



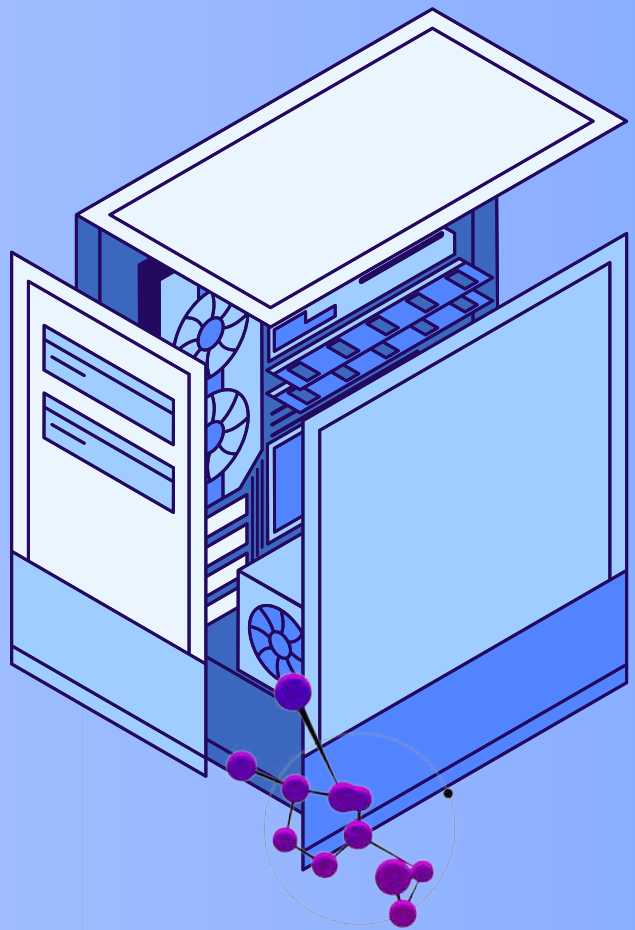
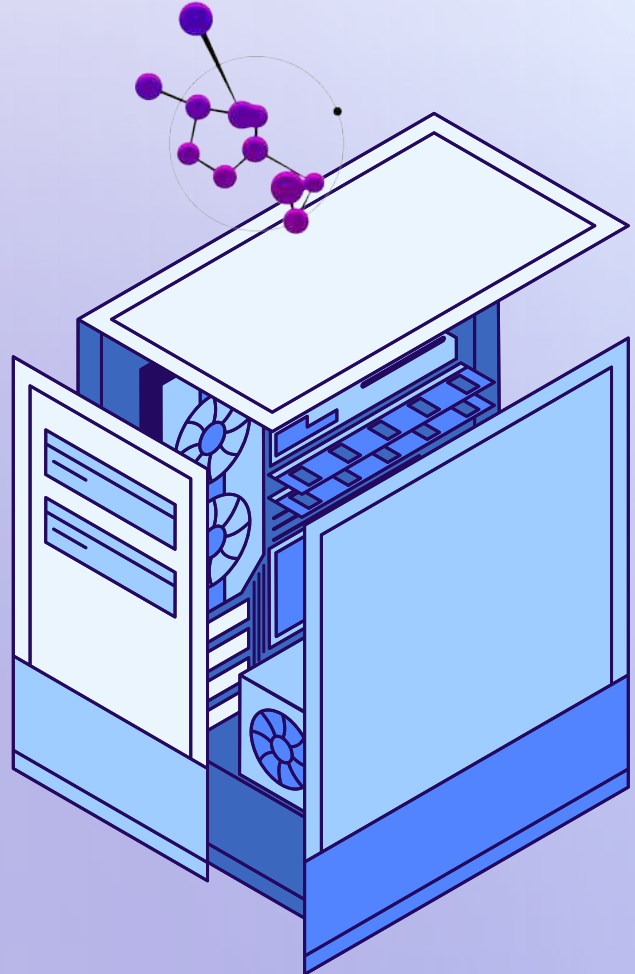
Clemens Horn

BACHELORARBEIT - KOLLOQUIUM

**FUNKTIONALES PAKETMANAGEMENT
UND CONTAINERISIERUNGSTECHNOLOGIEN
IN SOFTWARE ENTWICKLUNG UND
SOFTWARE DEPLOYMENT**



MOTIVATION



Moderne
Softwareprojekte
werden zunehmend
komplexer.





DEPENDENCY HELL

Widersprüchliche
Abhängigkeiten führen zu
Konflikten zwischen
Softwarepaketen





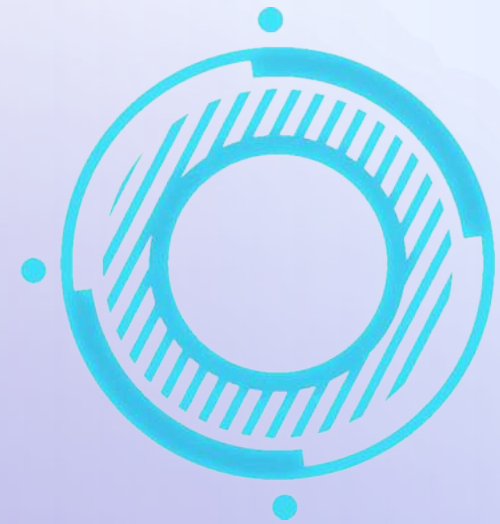
UMGEBUNGSDRIFT

Herausforderung,
die gleiche Konfiguration in
Entwicklung, Staging und
Produktion sicherzustellen





Einführung



KEINE KONSISTENZ

Probleme, die auftreten,
wenn Software auf einer
Maschine funktioniert, aber
nicht auf einer anderen





ÜBERBLICK THESIS

Vergleichende
Analyse von zwei
Hauptansätzen





FUNKTIONALES PAKETMANAGEMENT

Fokus auf
Reproduzierbarkeit
und deklarative
Konfiguration





CONTAINERISIERUNG

Fokus auf Portabilität
und Prozessisolierung





Einführung

FORSCHUNGSFRAGE



Wie vergleichen sich funktionales
Paketmanagement und
Containerisierungstechnologien als Entwicklungs-
und Bereitstellungstools, und welche
Empfehlungen lassen sich für Unternehmen
ableiten, die diese Technologien nutzen?



ZIELE THESIS

Unterschiede zwischen
funktionalem
Paketmanagement und
Containerisierung
verstehen.





ZIELE THESIS

Analyse der Stärken und
Schwächen in realen
Entwicklungsumgebungen





ZIELE THESIS

Empfehlungen für Entwickler
und Organisationen zur
Wahl des passenden
Ansatzes geben

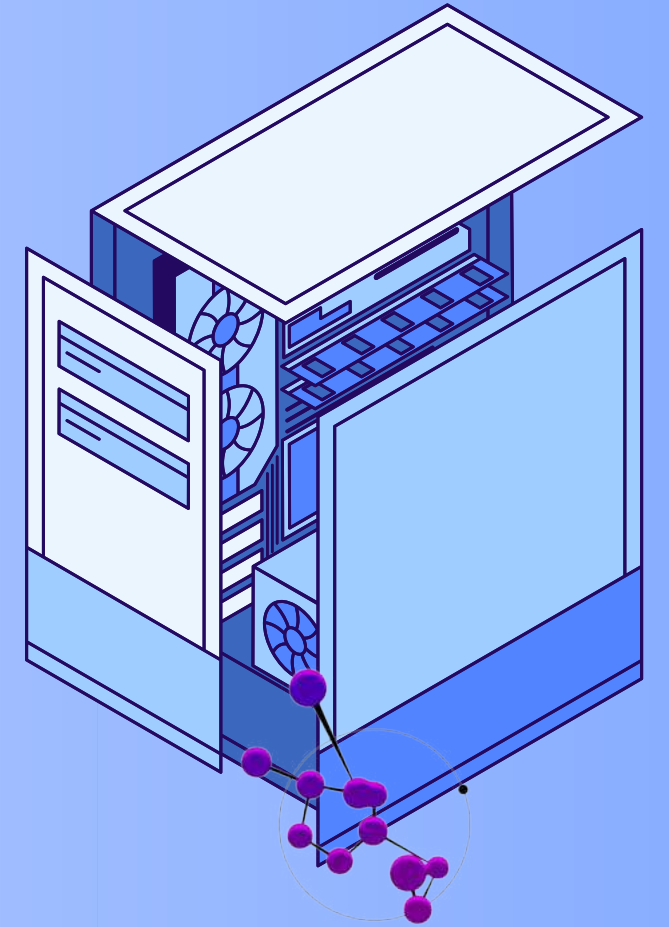
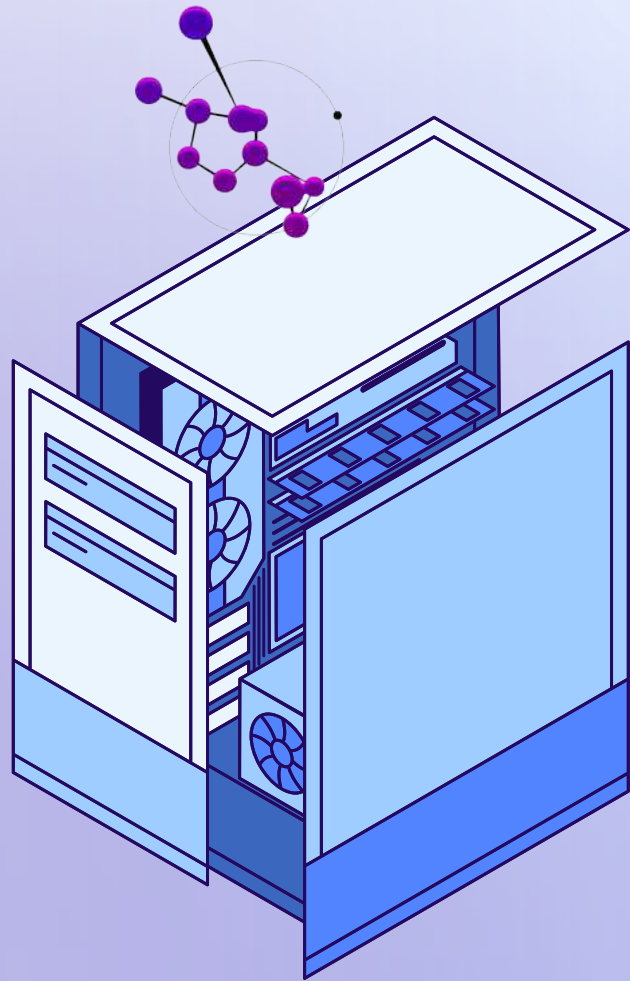




Einführung

STRUKTUR

Theoretische Grundlagen

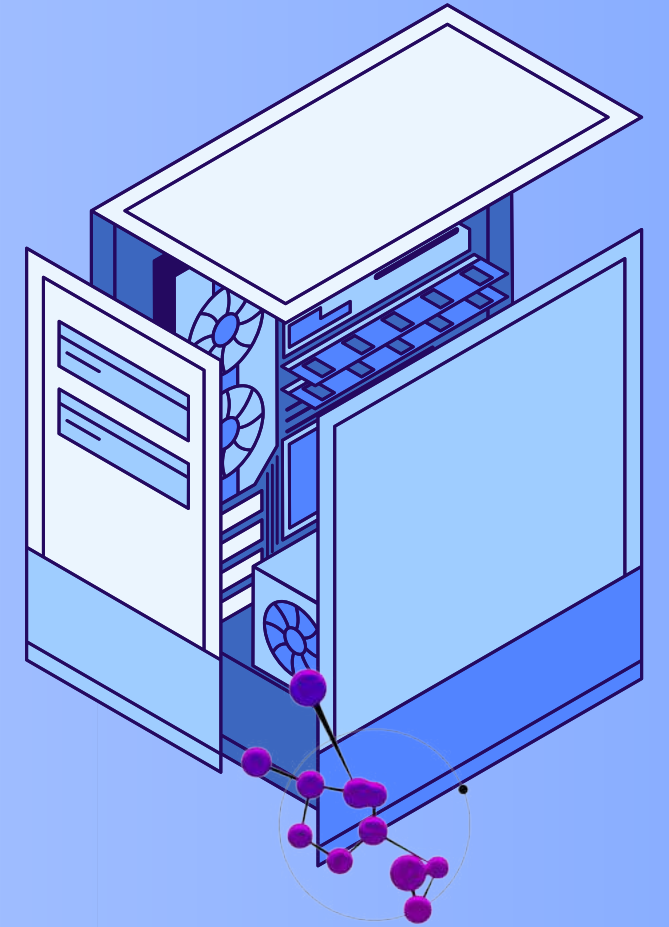
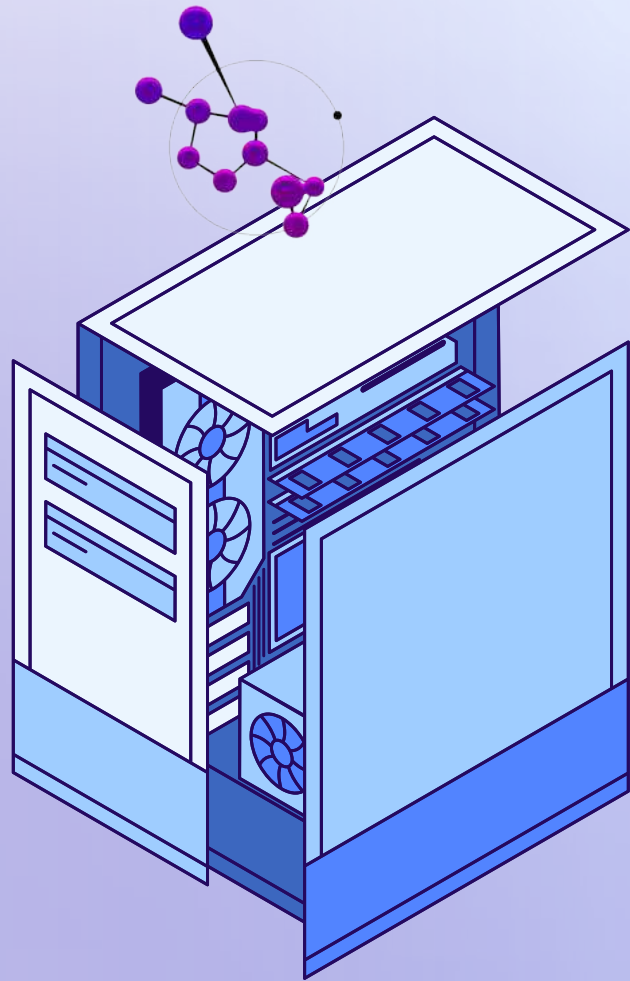




Einführung

STRUKTUR

Praktische
Anwendung

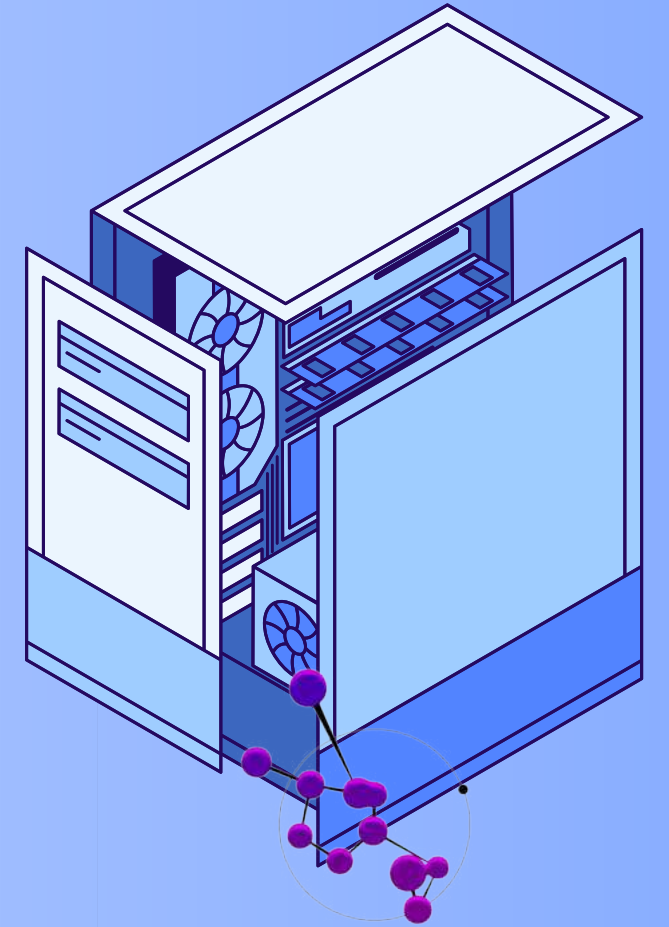
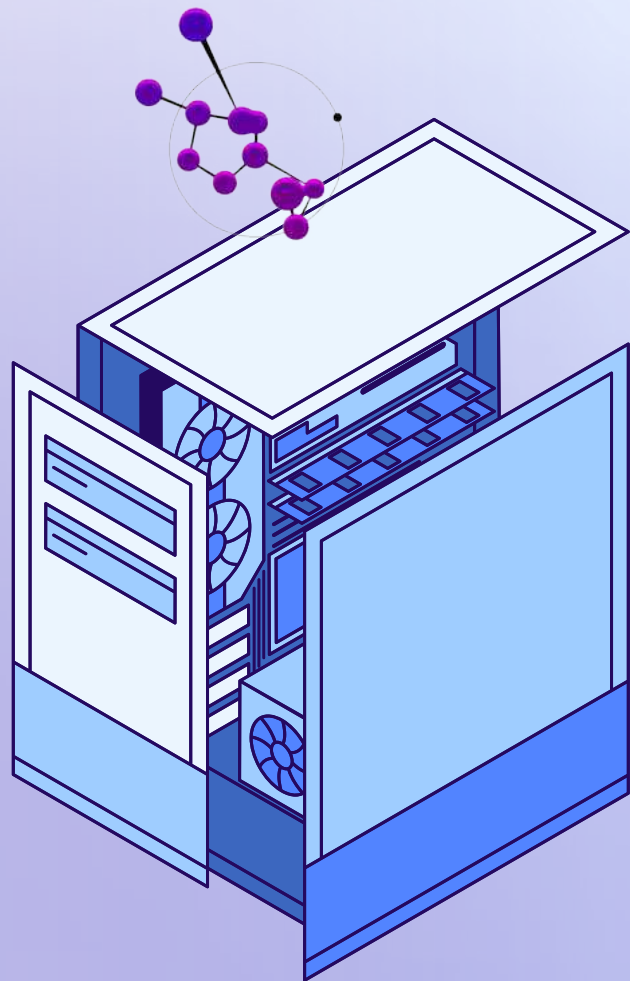




Einführung

STRUKTUR

Analyse &
Empfehlungen

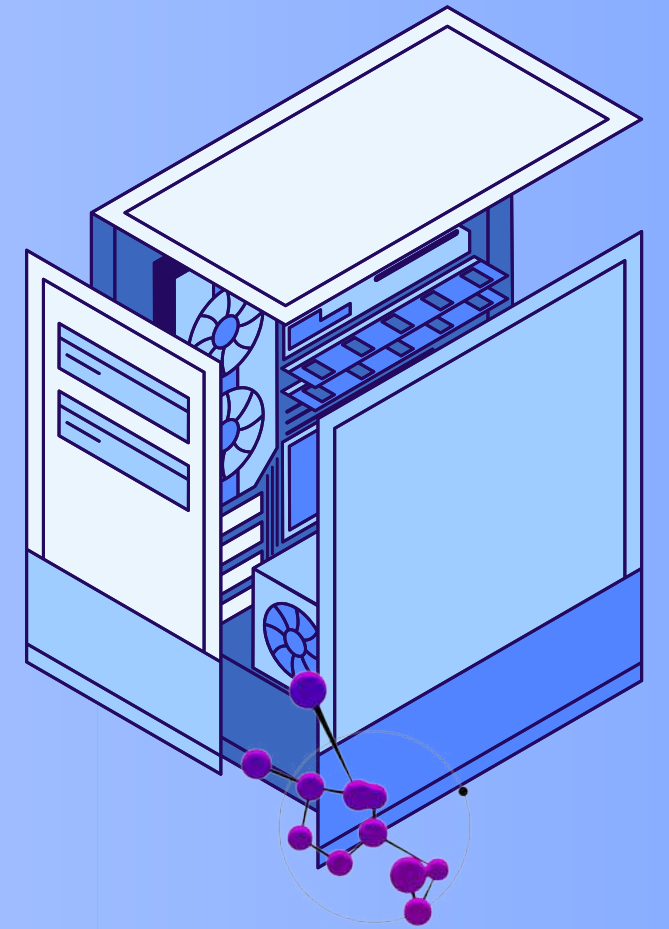
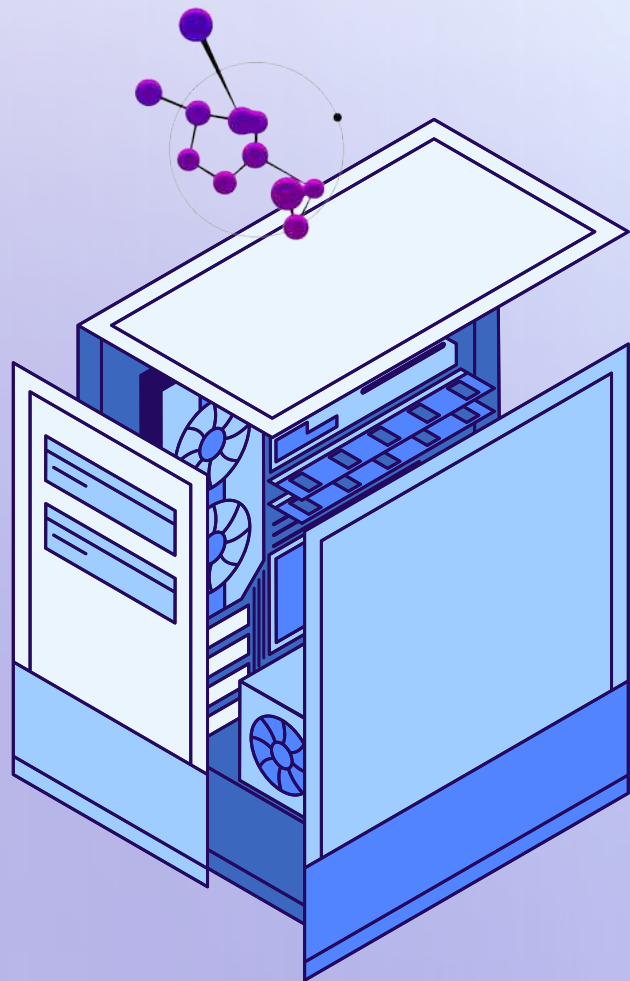




Einführung

STRUKTUR

Fazit





Theoretische Grundlagen
Funktionales Paketmanagement

DEFINITION



Funktionales Paketmanagement ist eine Methode zur Verwaltung von Softwarepaketen, bei der diese als unveränderliche Einheiten behandelt werden, um Reproduzierbarkeit sicherzustellen und deklarative Konfigurationsmethoden zu verwenden.





KONZEPTE IMMUTABILITÄT

Pakete sind unveränderlich,
sobald sie realisiert wurden





KONZEPTE

REPRODUZIERBARKEIT

Exakte Zustände können
jederzeit und auf jedem
System reproduziert werden





KONZEPTE

ABHÄNGIGKEITSISOLATION

Pakete werden voneinander isoliert, indem jegliche Abhängigkeiten gehasht werden, wobei jede Abhängigkeit samt der transitiven Inputs gehasht wird





KONZEPTE

DEKLARATIVE KONFIGURATION

Die gewünschte Konfiguration wird beschrieben, anstatt manuell imperativ installiert





Theoretische Grundlagen
Funktionales Paketmanagement

KONZEPTE STORE

Zentraler Speicherort
aller Paketversionen





KONZEPTE ROLLBACK

Möglichkeit der
Wiederherstellung
eines vorherigen
Zustands





Theoretische Grundlagen
Funktionales Paketmanagement

KONZEPTE CLOSURES

Abhängigkeiten sind
vollständig isoliert





BEISPIEL

NIX



Reine funktionale Ansätze

Nix-Ausdrücke definieren
den Bauprozess, das
Buildsystem und
Abhängigkeiten



Theoretische Grundlagen
Funktionales Paketmanagement

BEISPIEL

GNU GUIX



Basierend auf Nix

Verwendet Scheme als
Programmiersprache

Dialekt von Lisp



DEFINITION



Containerisierung ist eine leichtgewichtige Form der Virtualisierung, bei der eine Anwendung zusammen mit ihren Abhängigkeiten, Bibliotheken und Konfigurationsdateien in einem einzigen Container verpackt wird. Im Gegensatz zu herkömmlichen virtuellen Maschinen teilen sich Container den Kernel des Host-Betriebssystems, laufen jedoch als isolierte Prozesse.





Theoretische Grundlagen
Containerisierung

KONZEPTE NAMESPACES

Isolierung von Systemressourcen

Prozess-ID

Netzwerk

Dateisystem





Theoretische Grundlagen
Containerisierung

KONZEPTE

CONTROL GROUPS

Kontrolle und Limitierung von
Ressourcenverbrauch

CPU
Speicher





Theoretische Grundlagen
Containerisierung

KONZEPTE

PORTABILITÄT

Container laufen auf allen Systemen, die von der Container-Runtime unterstützt werden

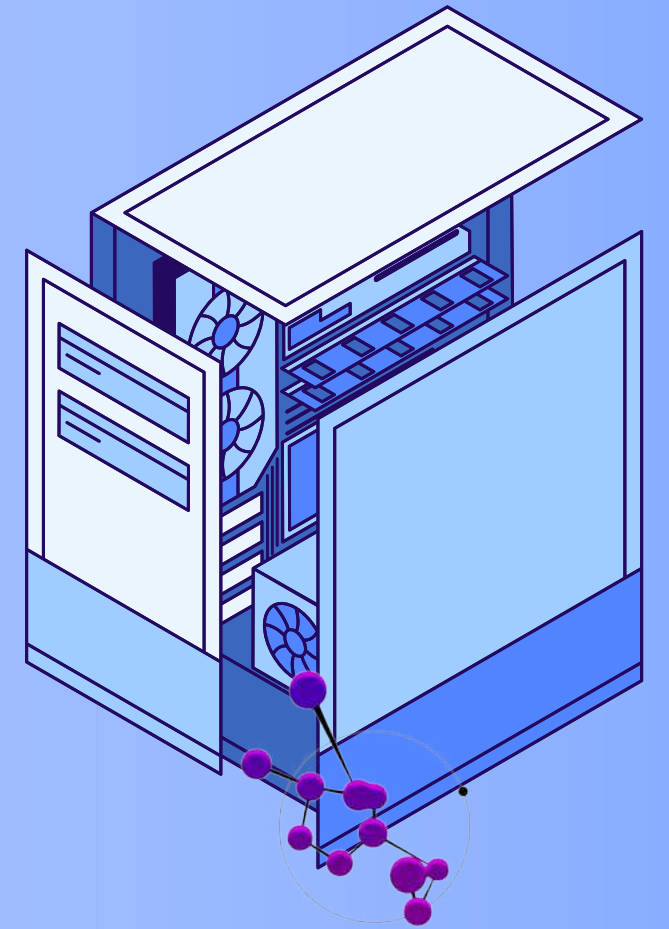
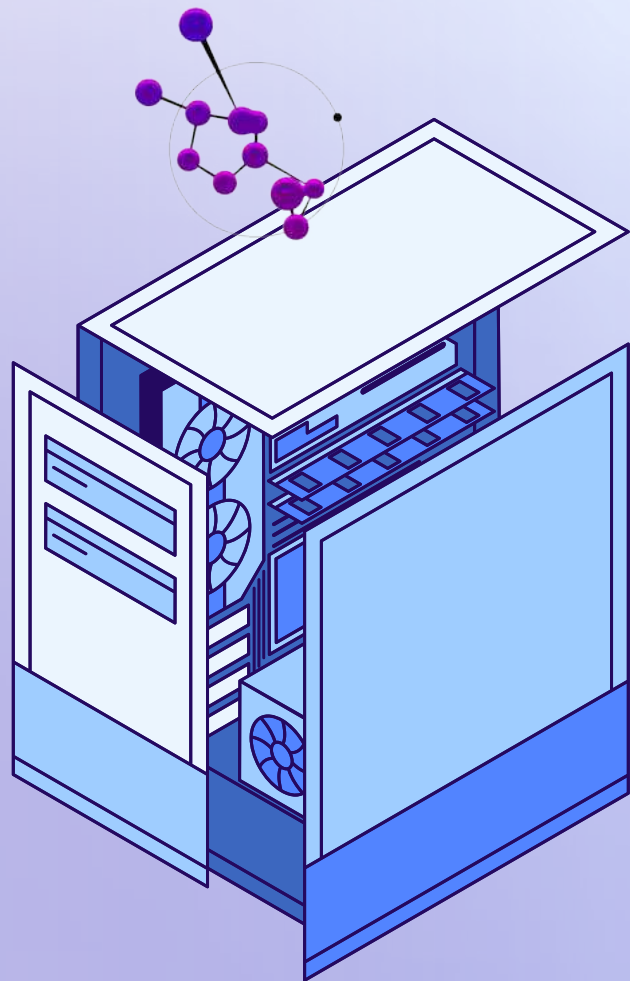




KONZEPTE

Images

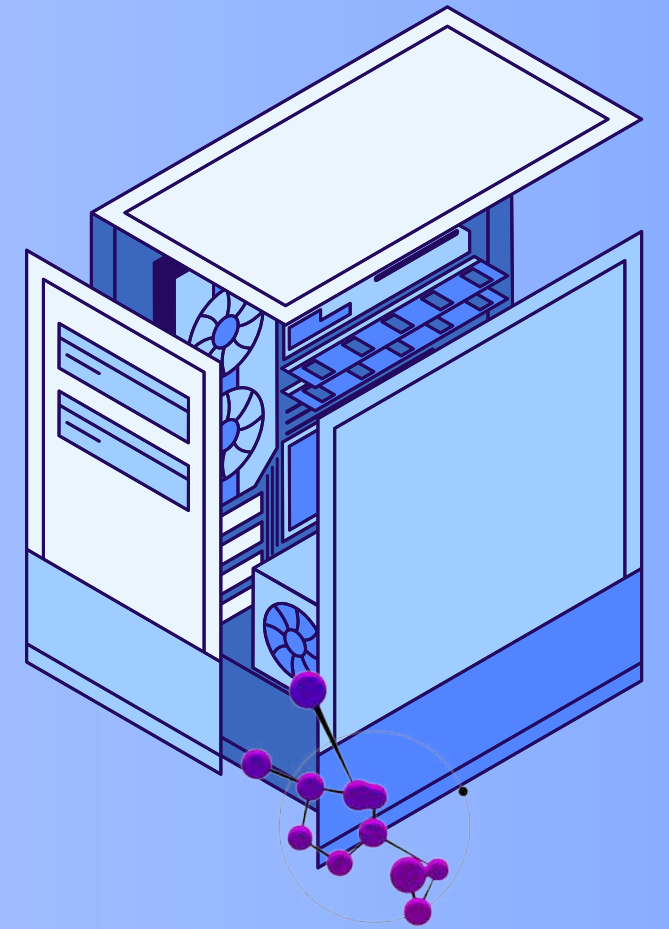
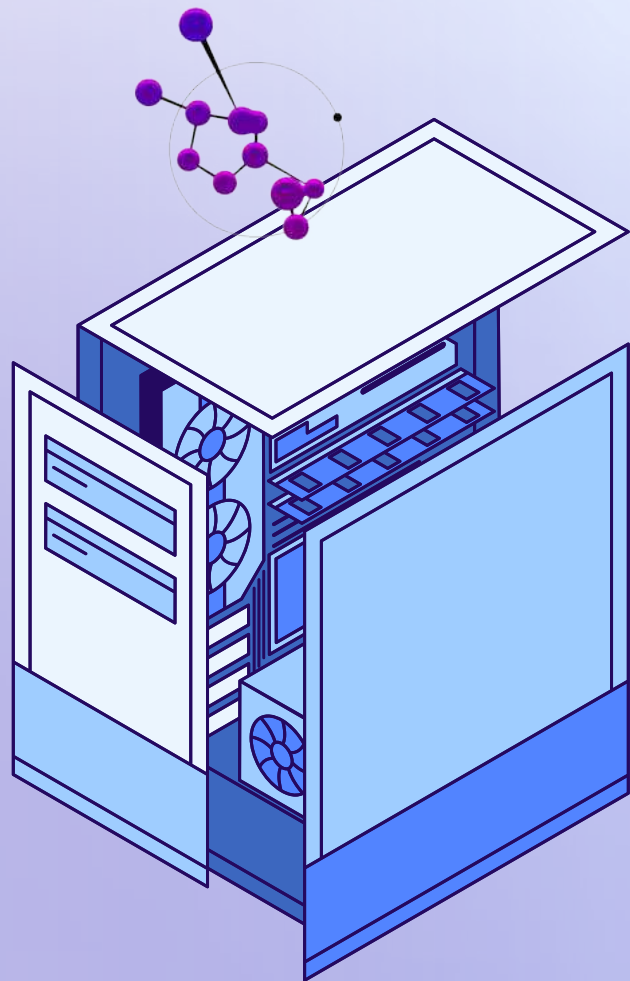
Selbstenthaltende
Softwarepakete mit
allem, was zur
Ausführung nötig ist





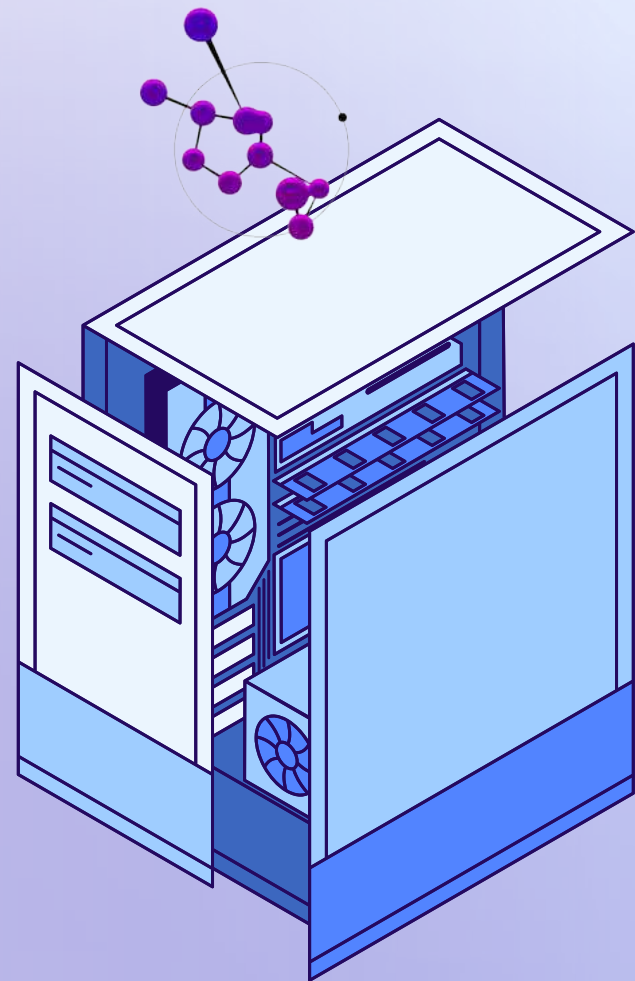
KONZEPTE

Layering
Reduzierung von
Redundanzen und
schnellere Builds



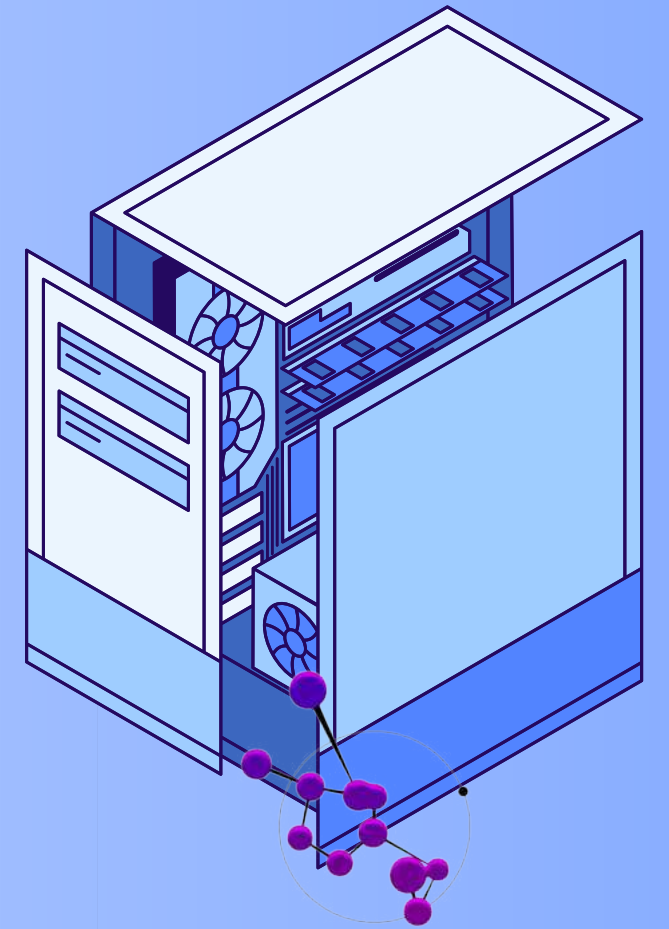


KONZEPTE



Runtime

Verwaltung des
Container-Lebenszyklus
(CRI-O, containerd)





Theoretische Grundlagen
Containerisierung

BEISPIEL

DOCKER



docker®

Beliebteste Plattform

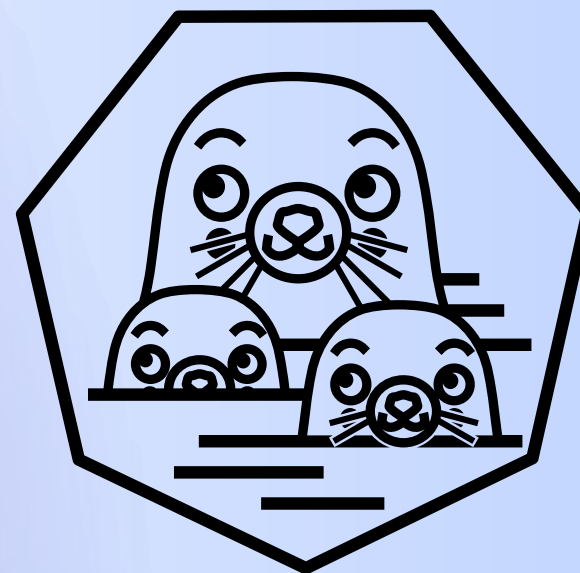
Einfache Erstellung und
Verwaltung von
Containern



Theoretische Grundlagen
Containerisierung

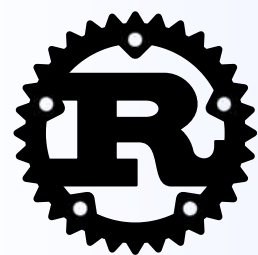
BEISPIEL

PODMAN



Daemonlos

Erhöhte Sicherheit durch
Rootless-Containers



Praktische Anwendung

ANWENDUNGSBEISPIEL



Demonstration der Integration von funktionalem
Paketmanagement und Containerisierung

Fokus auf Reproduzierbarkeit der
Entwicklungsumgebung und Effizienz der
Deployments





Praktische Anwendung - Nix

SETUP ENTWICKLUNGSUMGEBUNG

Installation von des
Paketmanagers namens Nix

In Windows via WSL





Praktische Anwendung - Docker

SETUP ENTWICKLUNGSUMGEBUNG

Docker Desktop auf Windows

Dev Containers in VSCode





Praktische Anwendung - Nix

ENTWICKLUNGSUMGEBUNG KONFIGURATION

Definition von Inputs in flake.nix

Definiere Outputs aus Inputs

Alle notwendigen Abhängigkeiten in
der Shell





ENTWICKLUNGSUMGEBUNG KONFIGURATION

Erstellung von Dockerfile

Integration in VSCode per
devcontainer.json





Praktische Anwendung - Nix

REPRODUZIERBARE BUILDS

Abhängigkeiten werden
gehasht

Immutable Pakete





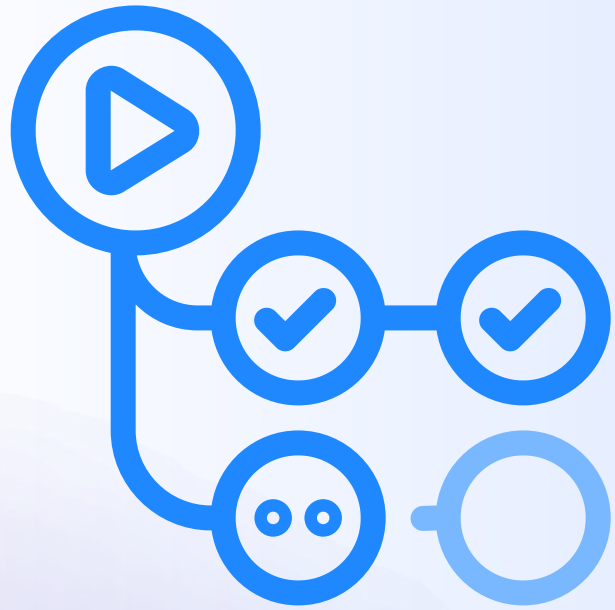
Praktische Anwendung - Docker

REPRODUZIERBARE BUILDS



Image-Tagging

Pinning von Versionen



Praktische Anwendung

CI/CD



Automatisierte Workflows für Builds, Tests und Deployments als Grundlage für Metrikenerfassung

Nachvollziehbare und reproduzierbare
Hardwareumgebung



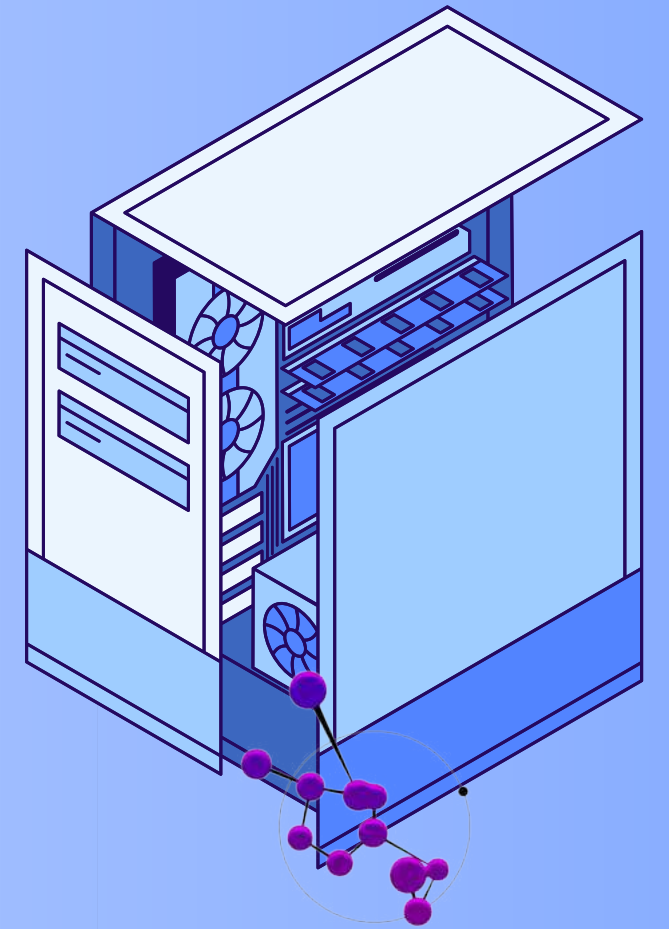
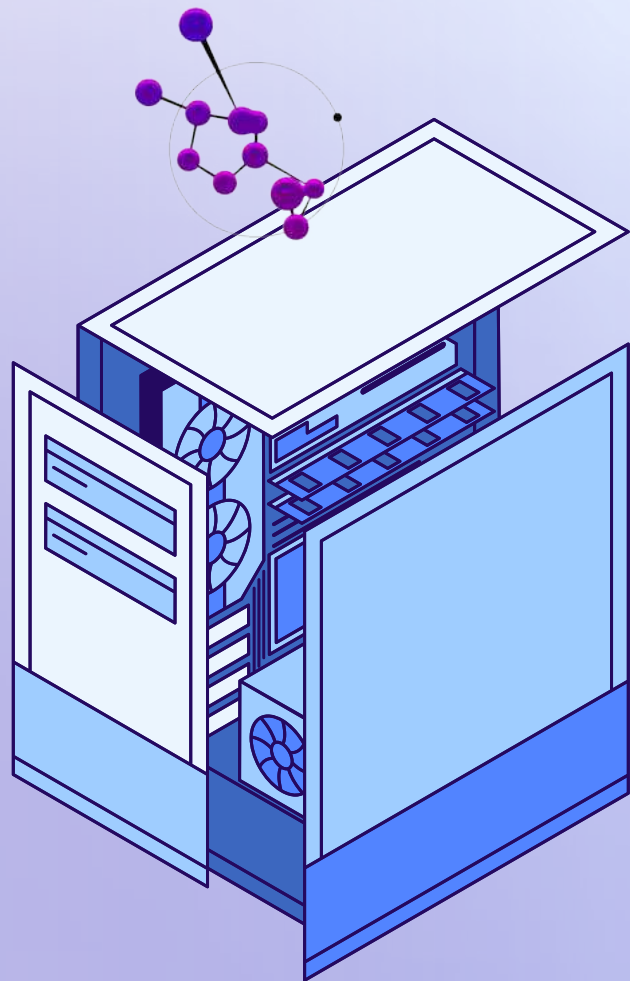


Praktische Anwendung

CI/CD

Binary Caching

Schnellere Builds



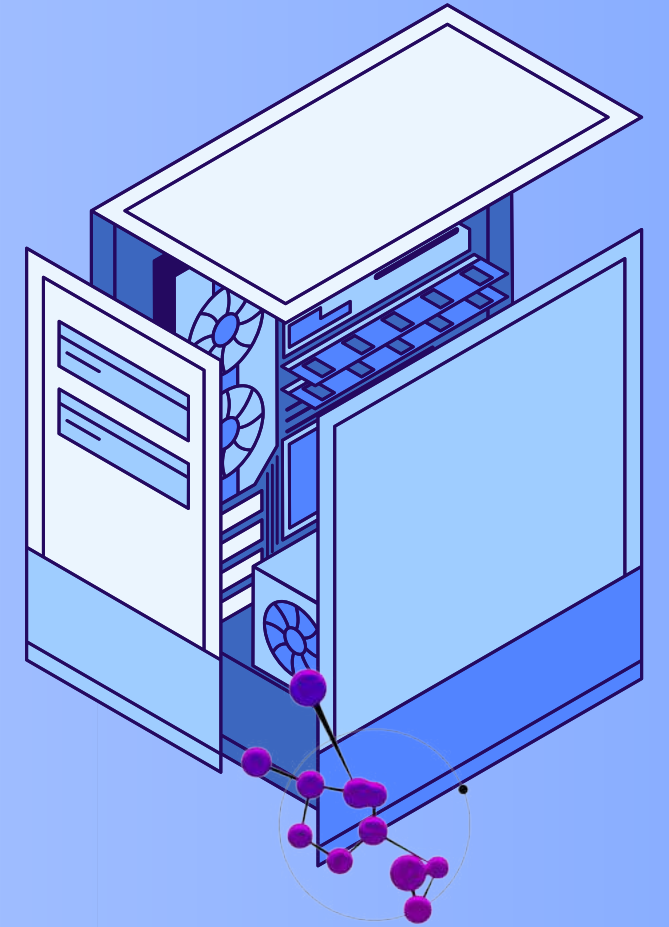
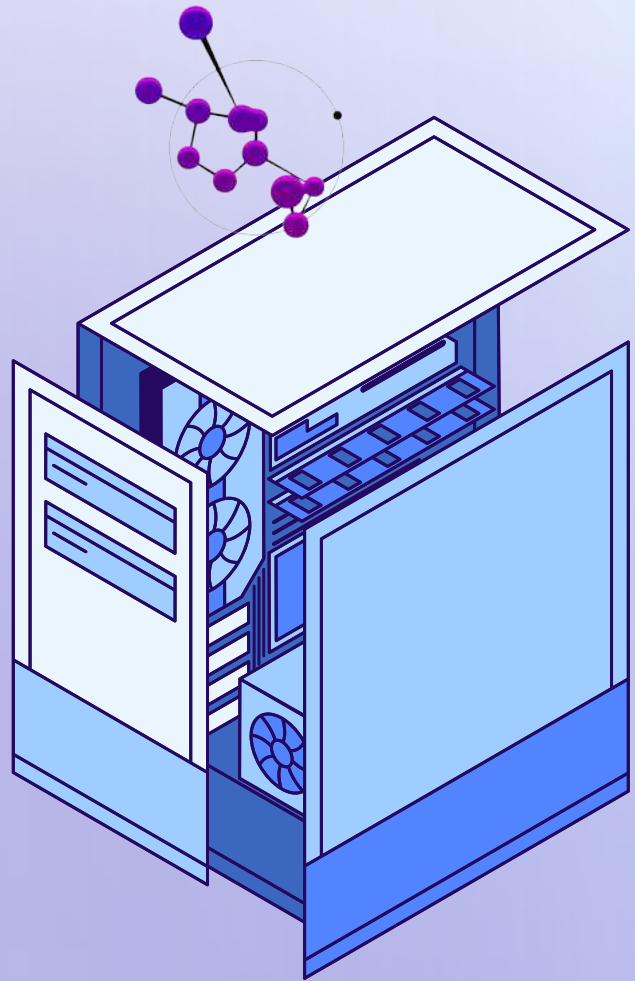


Praktische Anwendung

CI/CD

Multi-Stage-Builds

Images verbrauchen
weniger Speicher





Analyse

ERGEBNISSE



Vergleich der Effizienz von funktionalem
Paketmanagement und Containerisierung

Build-Zeiten

Paketgrößen

Entwicklungsumgebungen

Prozessmanagement

Software Deployments

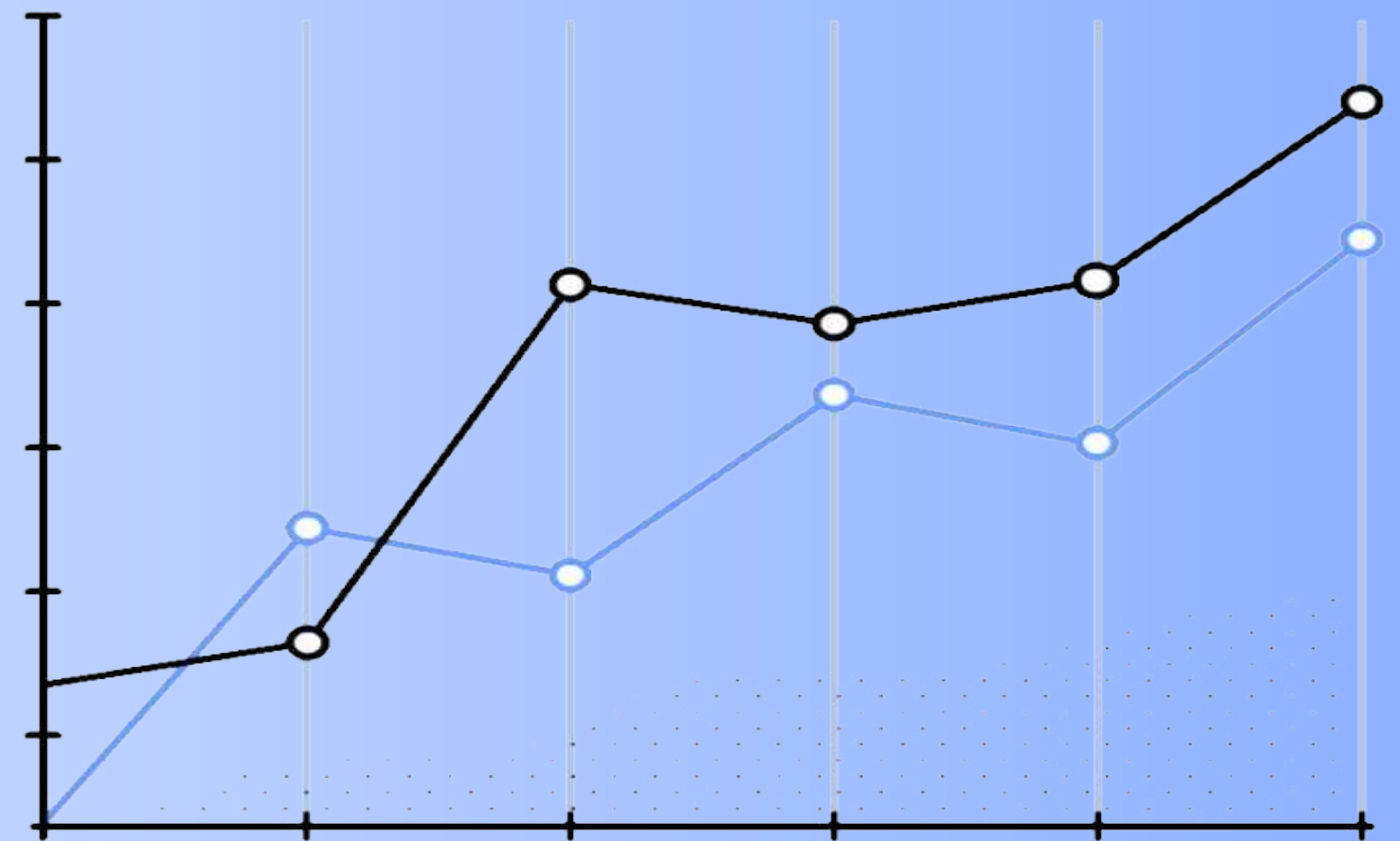


BUILD-ZEITEN

DEFINITION

Gesamtzeit des Build-Prozesses von Start bis Abschluss

Wichtiger Indikator für die Effizienz in CI/CD-Umgebungen

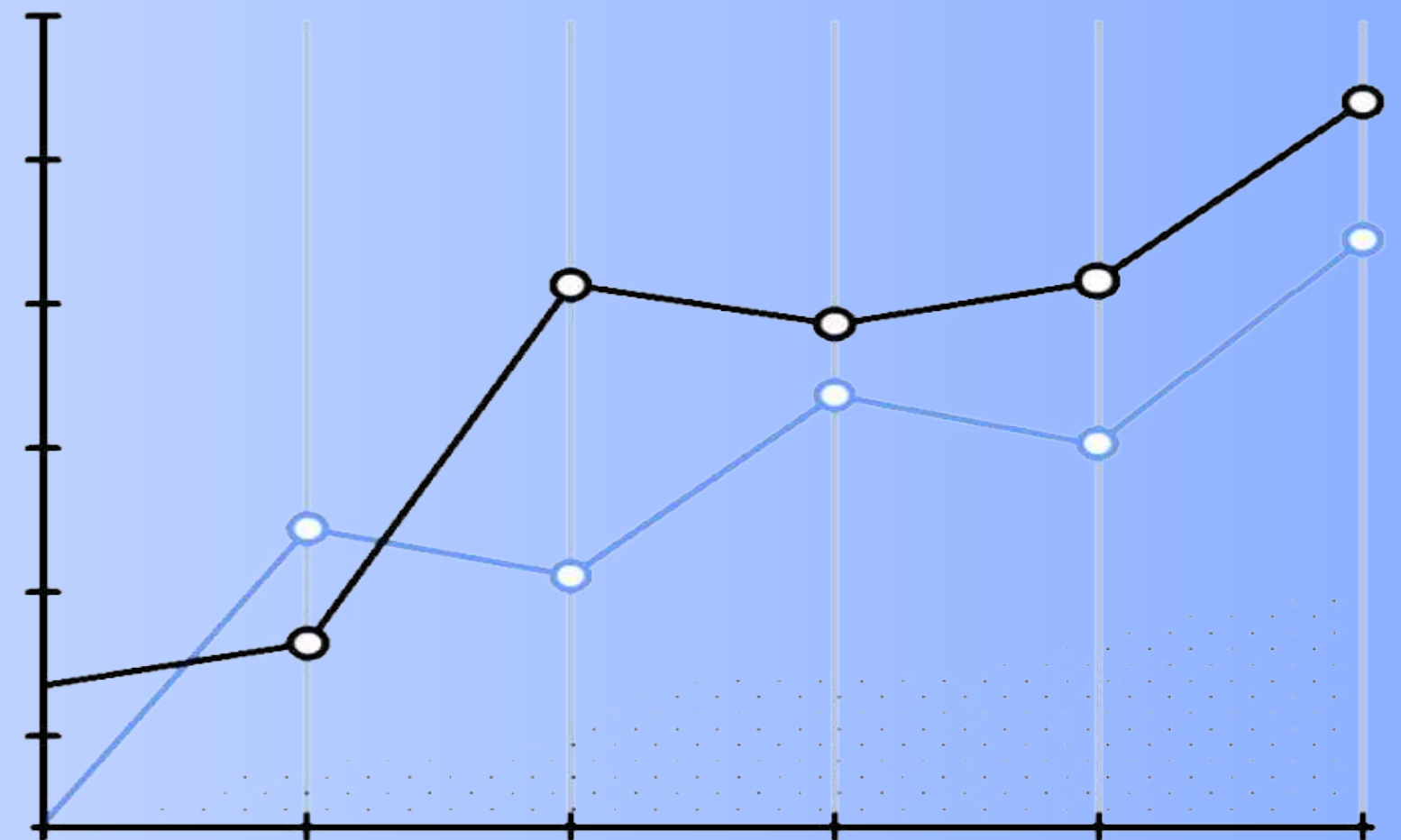


BUILD-ZEITEN

CONTAINER

Langsamer bei Cold Builds aufgrund der Abhängigkeit von Basis-Images

Signifikante Optimierung bei Warm Builds durch Layercache

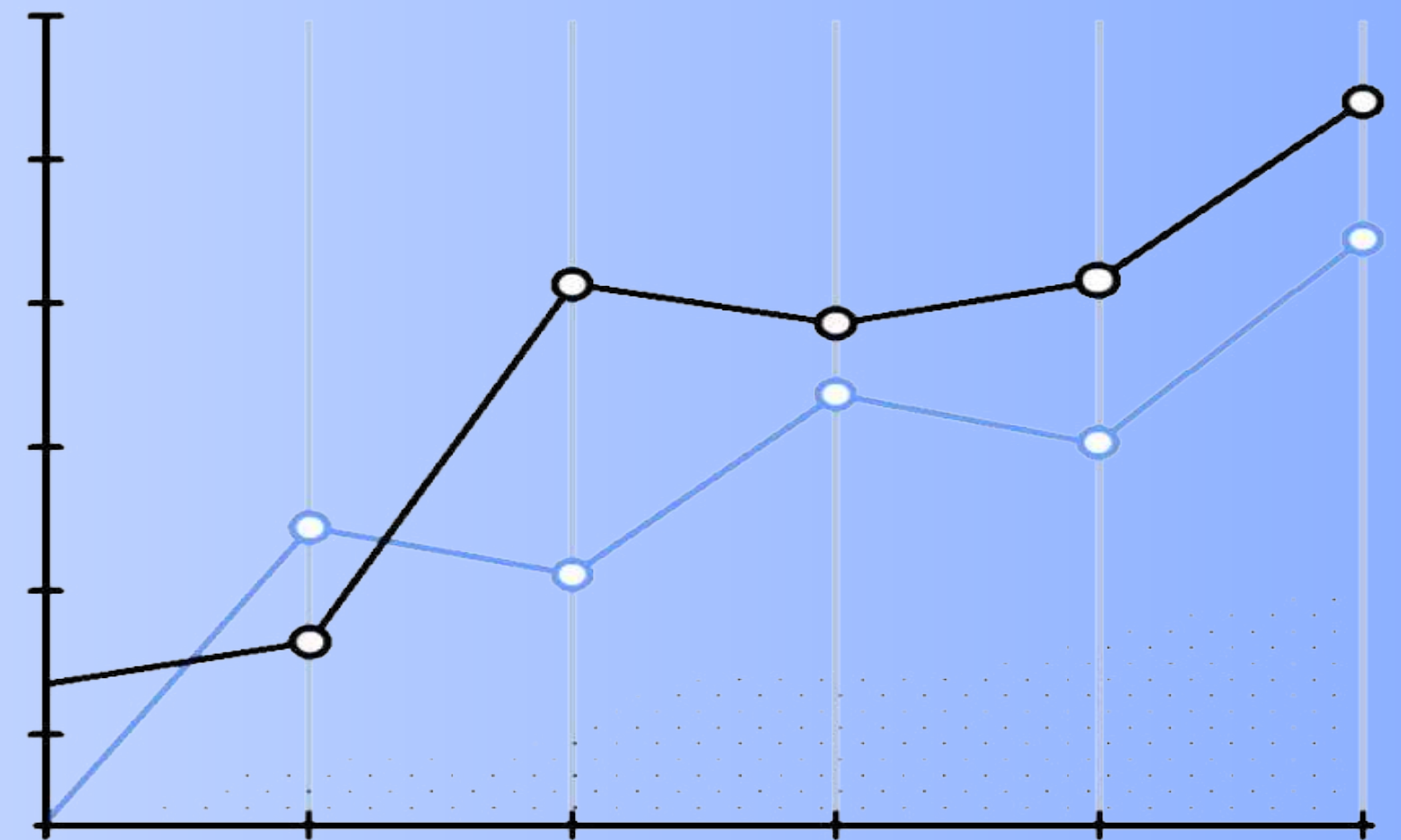


BUILD-ZEITEN

DERIVATIONEN

Schneller bei Cold-Builds

Weniger Performance-Gewinne bei Warm Builds aufgrund der Immutabilität

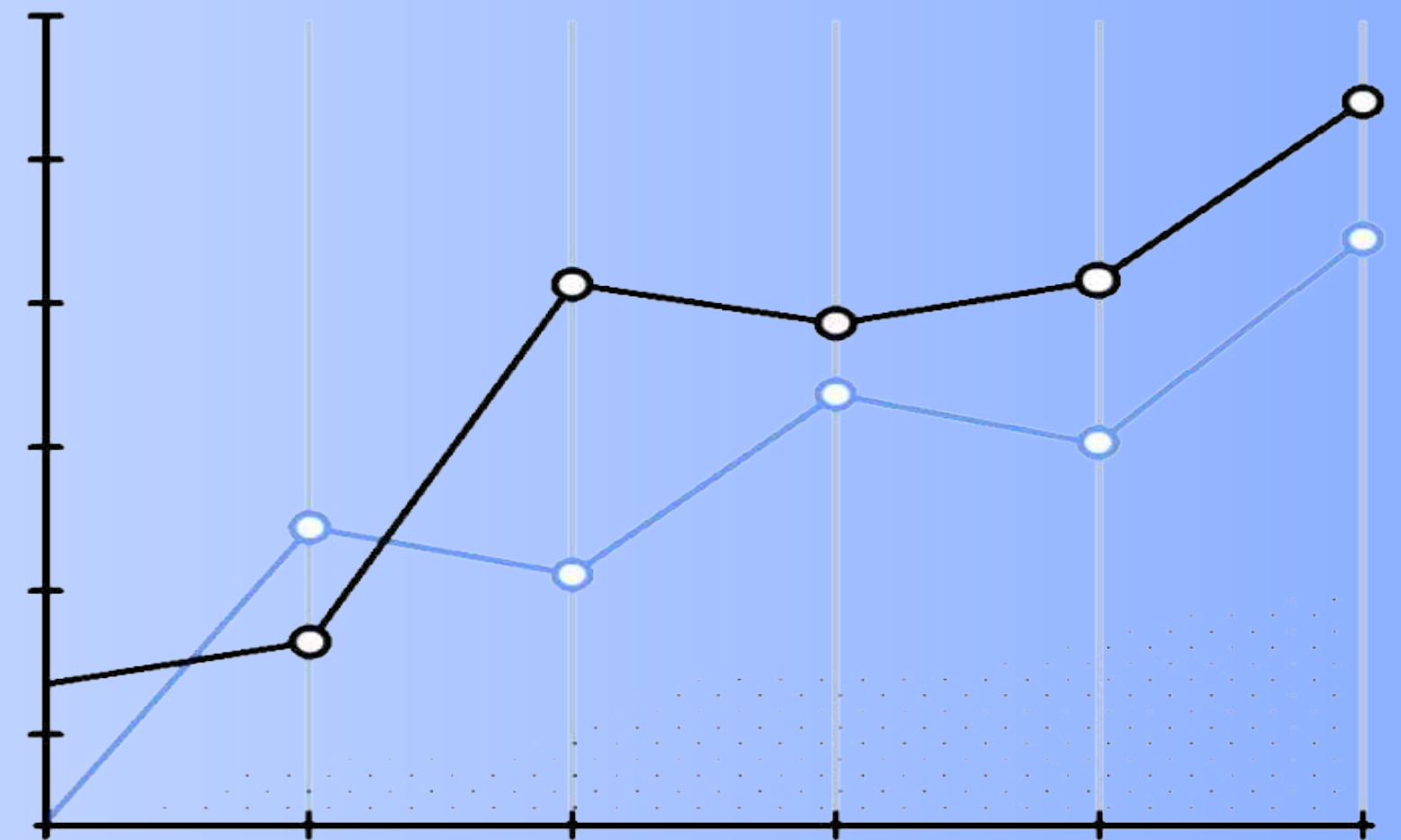


BUILD-ZEITEN

ERGEBNIS

Funktionales Paketmanagement übertrifft Containerisierung bei Cold Builds

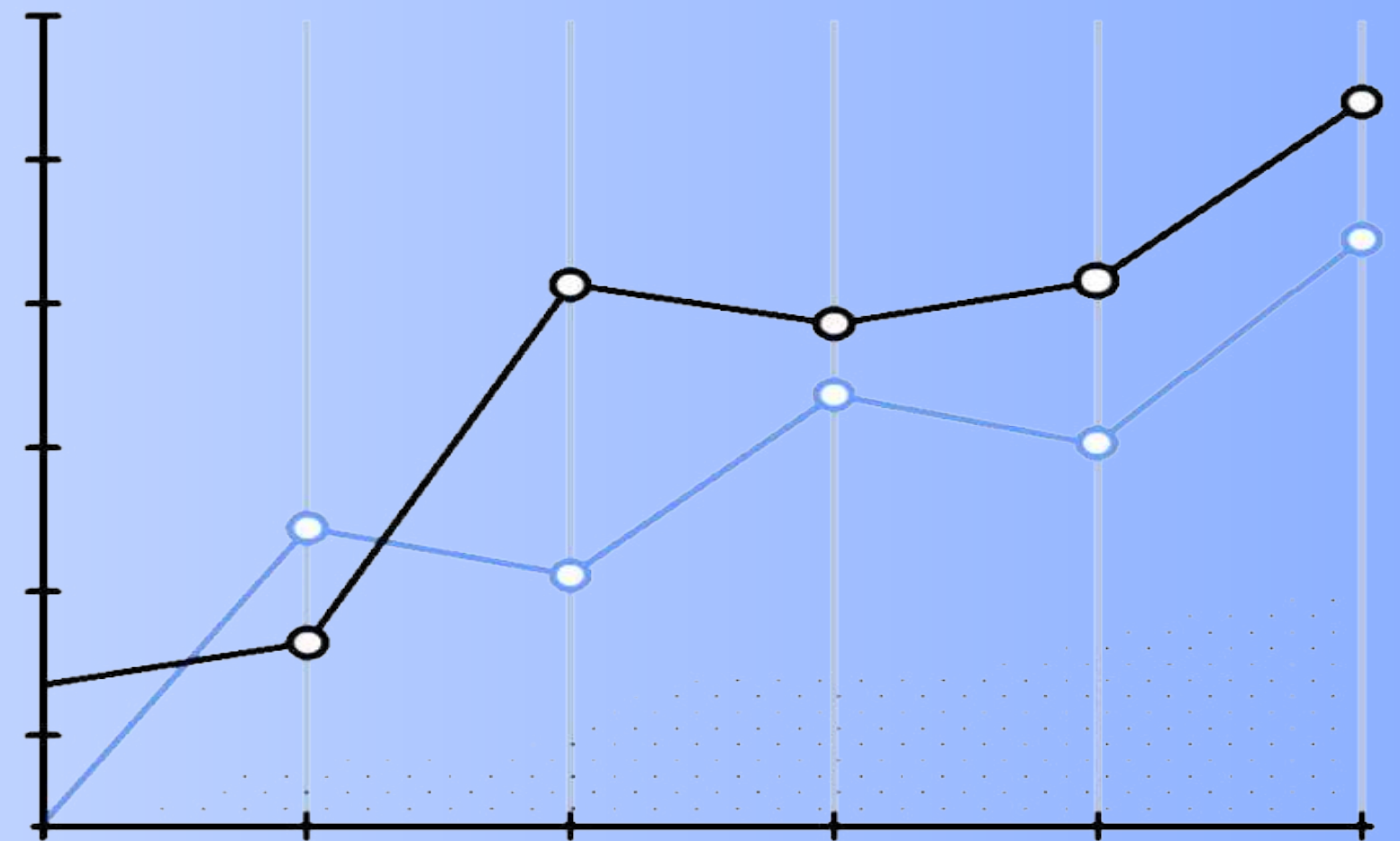
Containerisierung gewinnt in Warm Builds durch effiziente Caching-Strategien



SPEICHER CONTAINER

Enthalten das gesamte Betriebssystem
und alle Abhängigkeiten

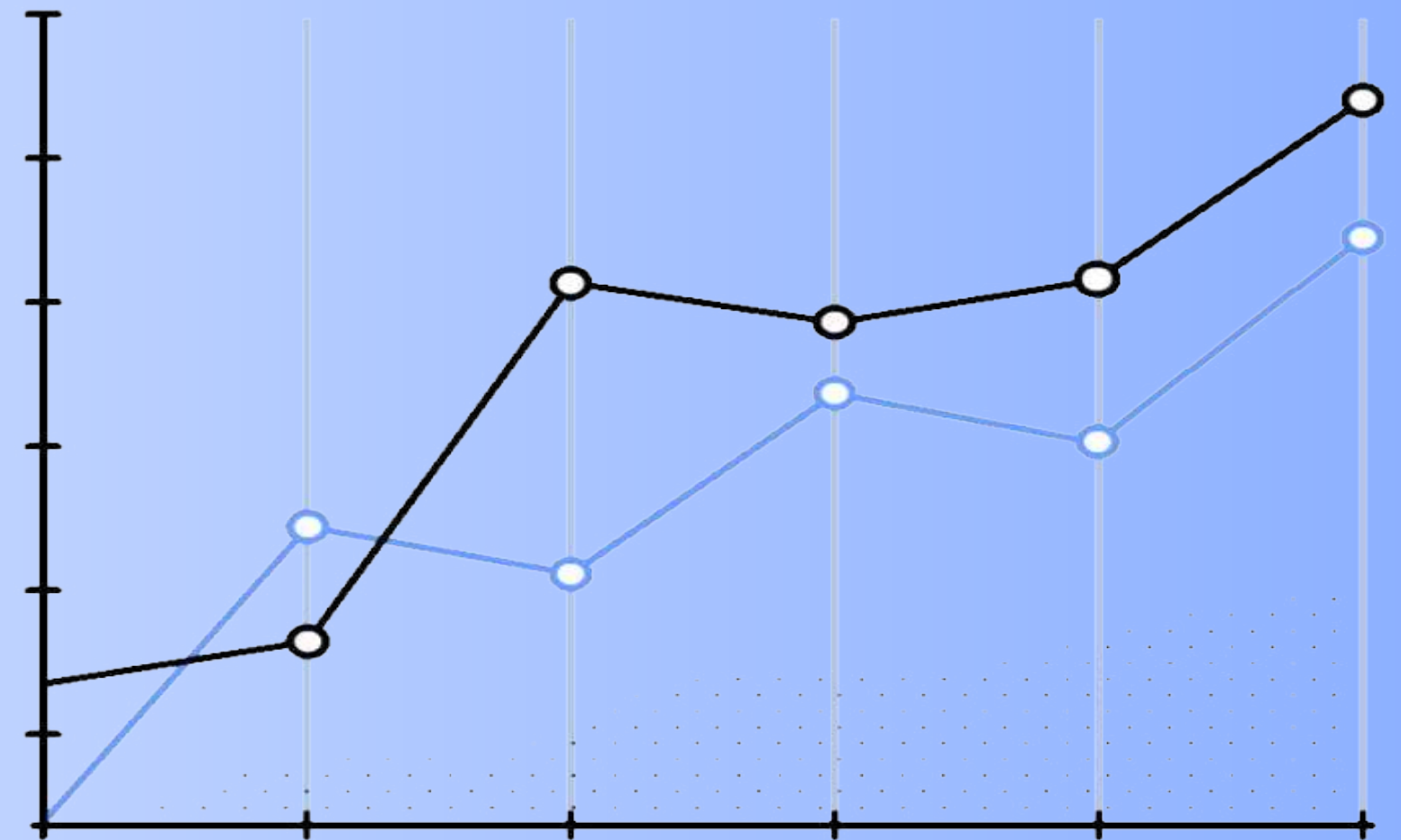
Optimierte Beispielanwendung
verbraucht 31,82 MB



SPEICHER DERIVATIONEN

Enthalten nur Laufzeitabhängigkeiten

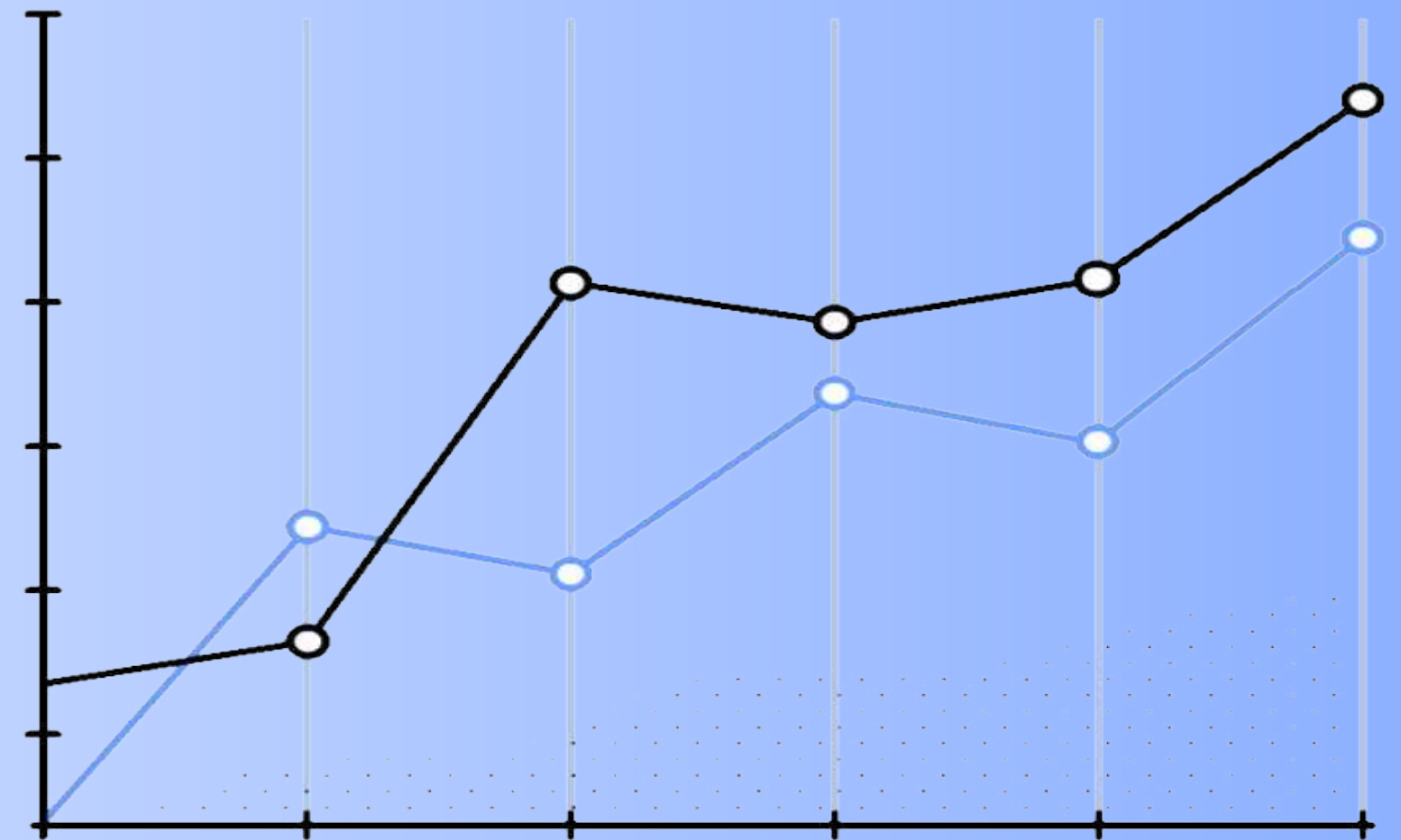
Optimierte Beispielanwendung
verbraucht lediglich 2,78 MB



SPEICHER ERGEBNIS

Funktionale Paketmanagement-Systeme erzeugen signifikant kleinere Pakete

Vorteilhaft für schnellere Deployments und geringeren Netzwerkverbrauch

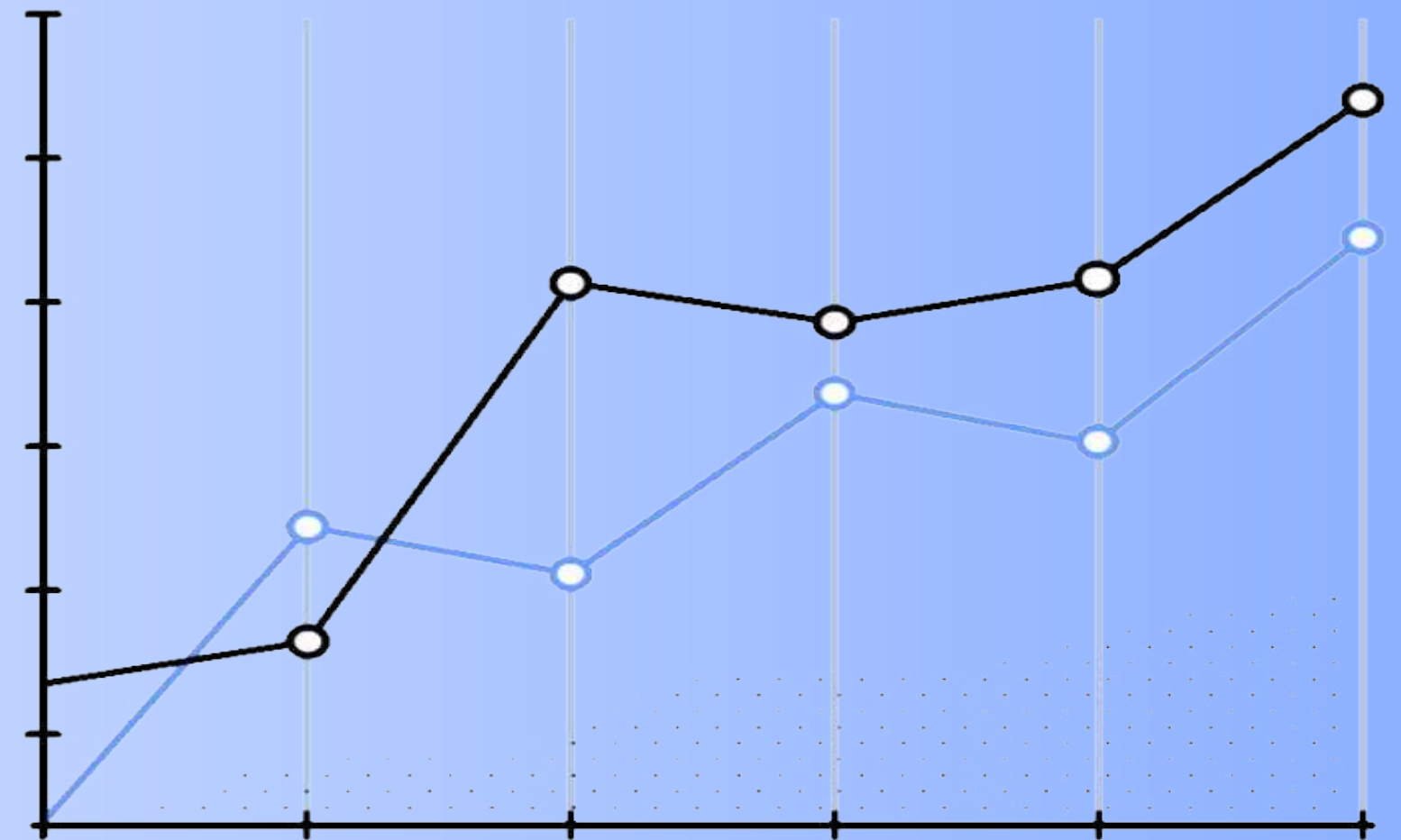


ENTWICKLUNG CONTAINER

Kapselung der gesamten Umgebung
einschließlich Betriebssystem und Tools

Keine Persönliche Konfiguration

Höherer Overhead auf Nicht-Linux-Systemen

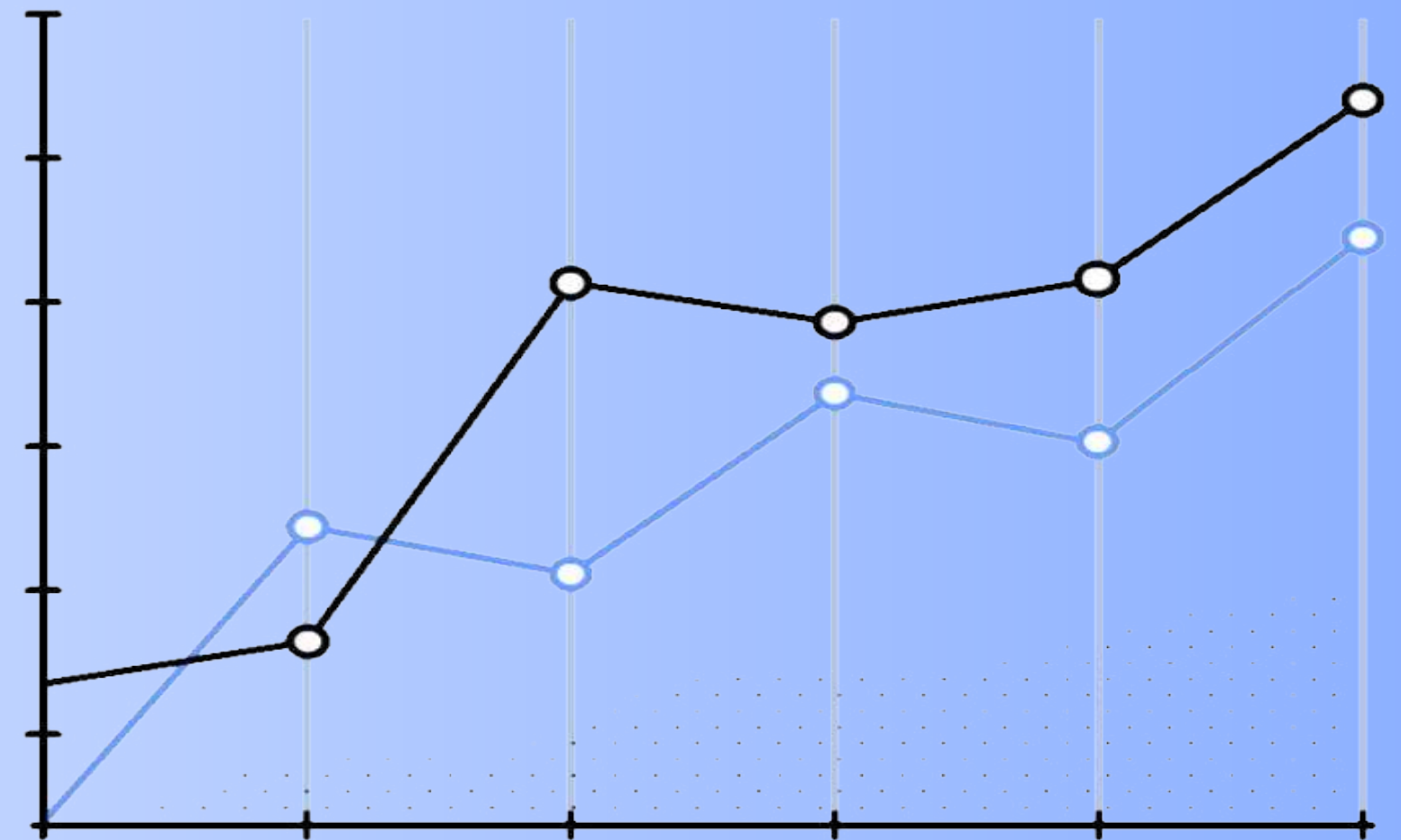


ENTWICKLUNG CONTAINER

Kapselung der gesamten Umgebung
einschließlich Betriebssystem und Tools

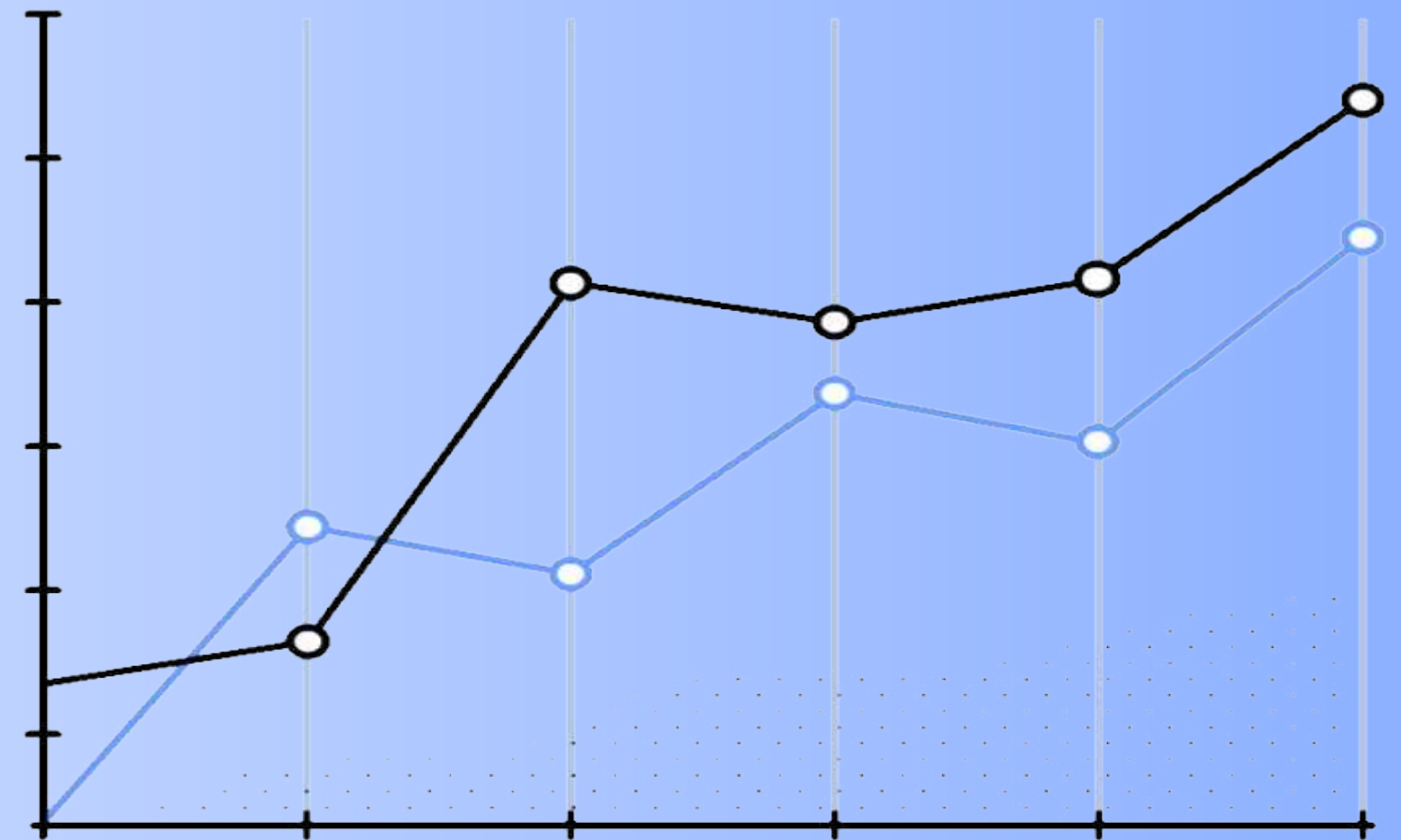
Keine Shell-Konfiguration

Höherer Overhead auf Nicht-Linux-Systemen



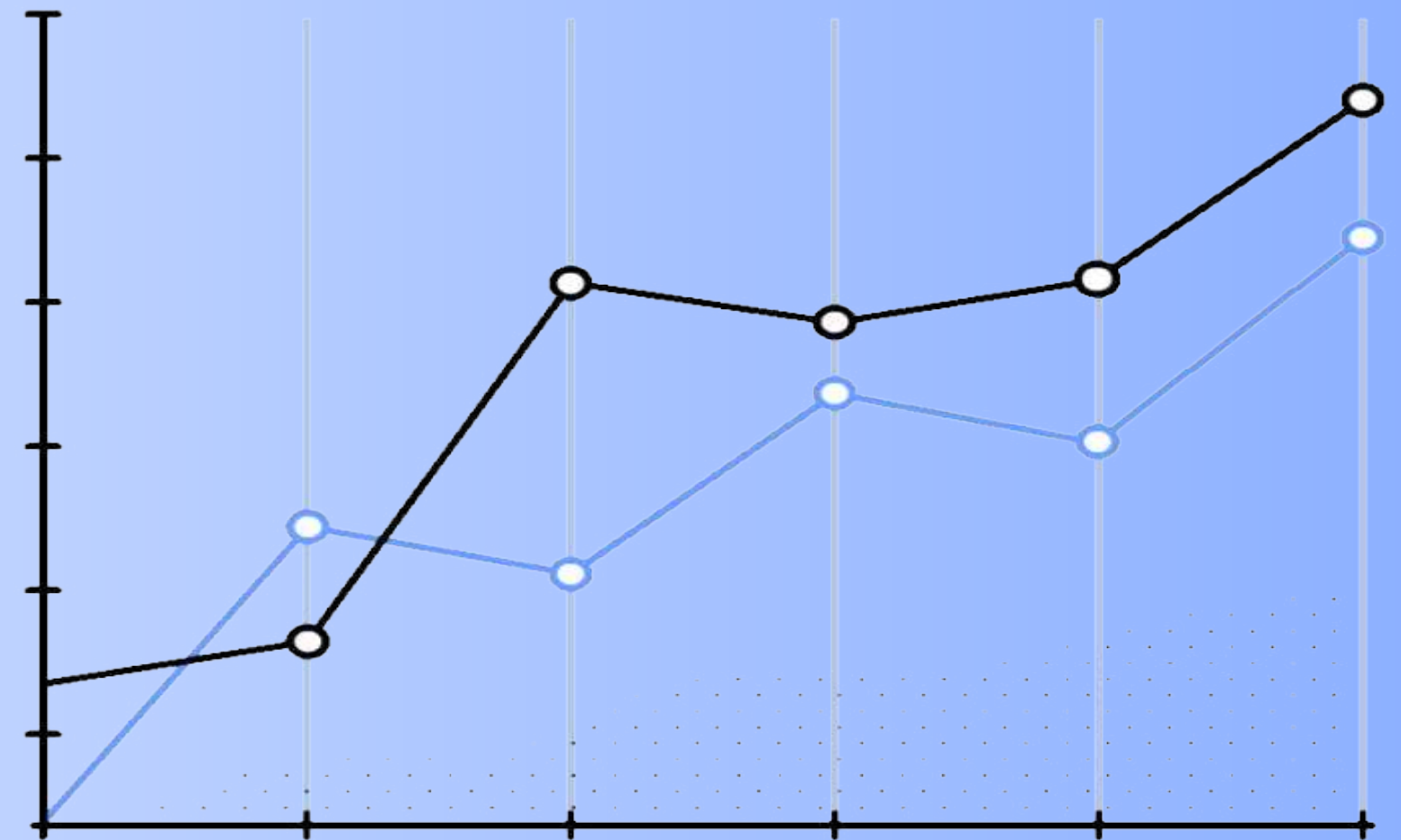
ENTWICKLUNG DERIVATIONEN

Deklarative Entwicklungsumgebungen
Vererbung der vorhandenen Shell
Binary Caching möglich
Nahtlose Hostintegration



ENTWICKLUNG ERGEBNIS

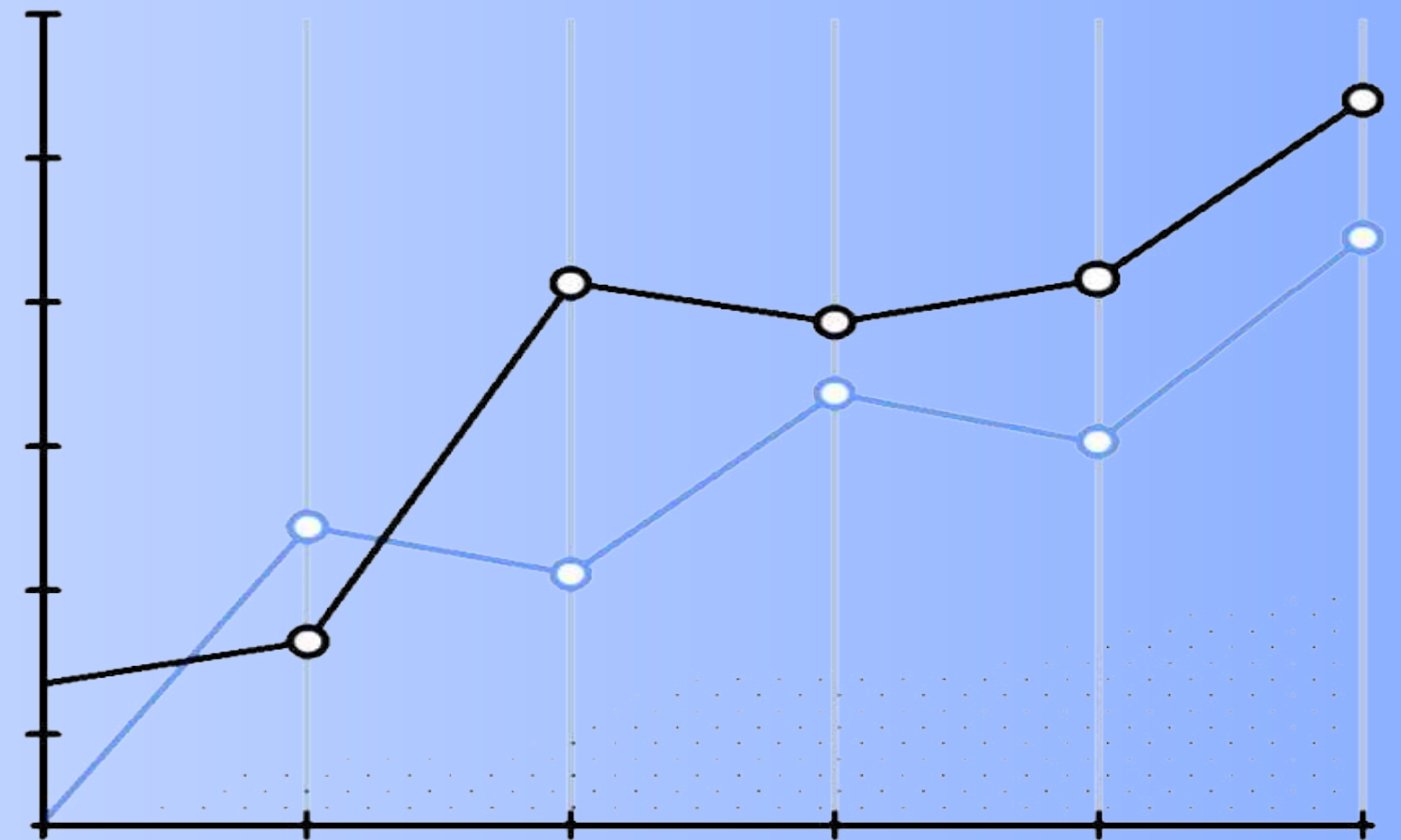
Containerisierung bietet mehr Isolation,
ist aber komplexer in der Verwaltung
Funktionales Paketmanagement
ermöglicht eine nahtlose Integration in
die Host-Umgebung



PROZESSE CONTAINER

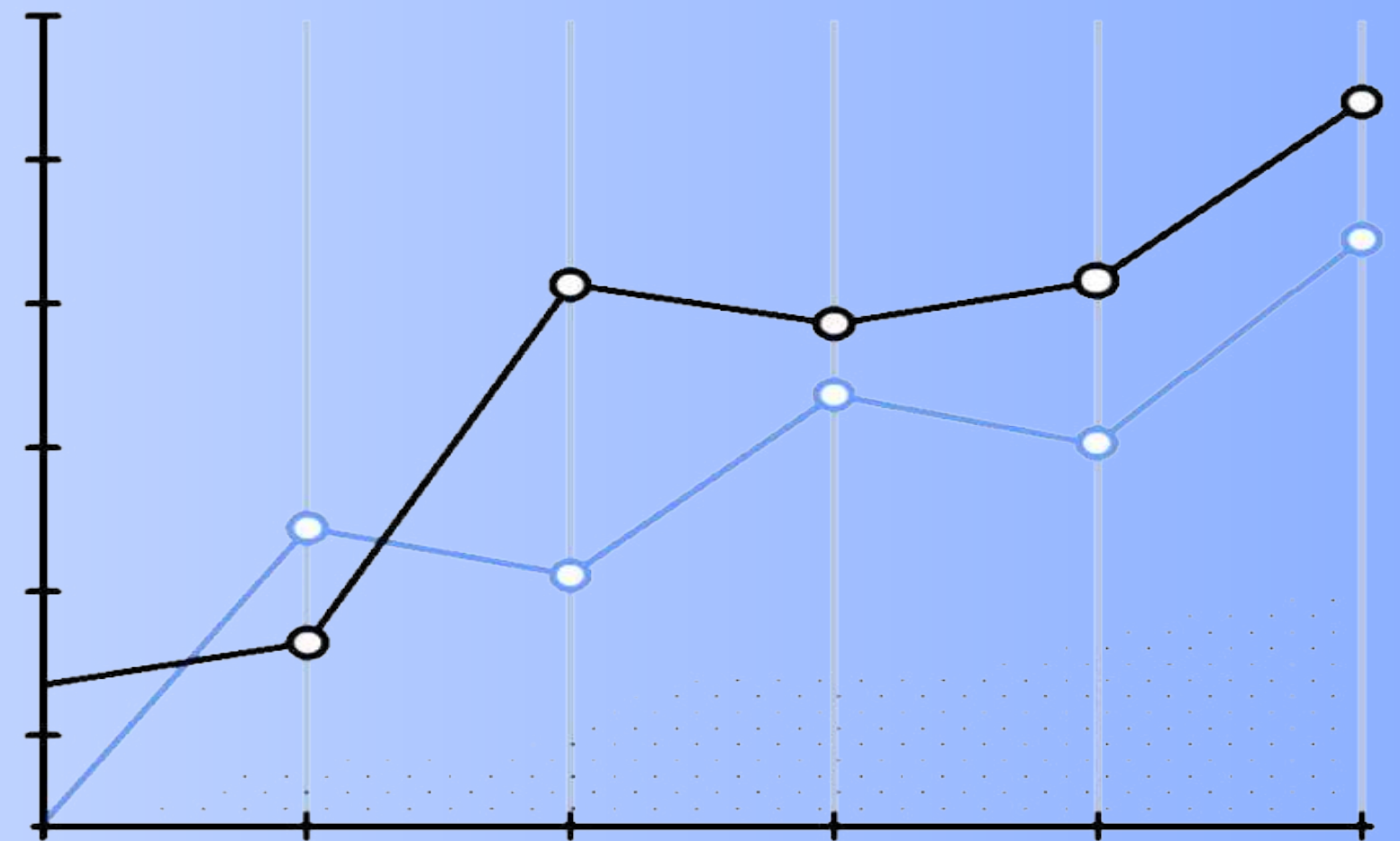
Docker Compose ermöglicht deklaratives
Prozessmanagement

Industriestandard



PROZESSE DERIVATIONEN

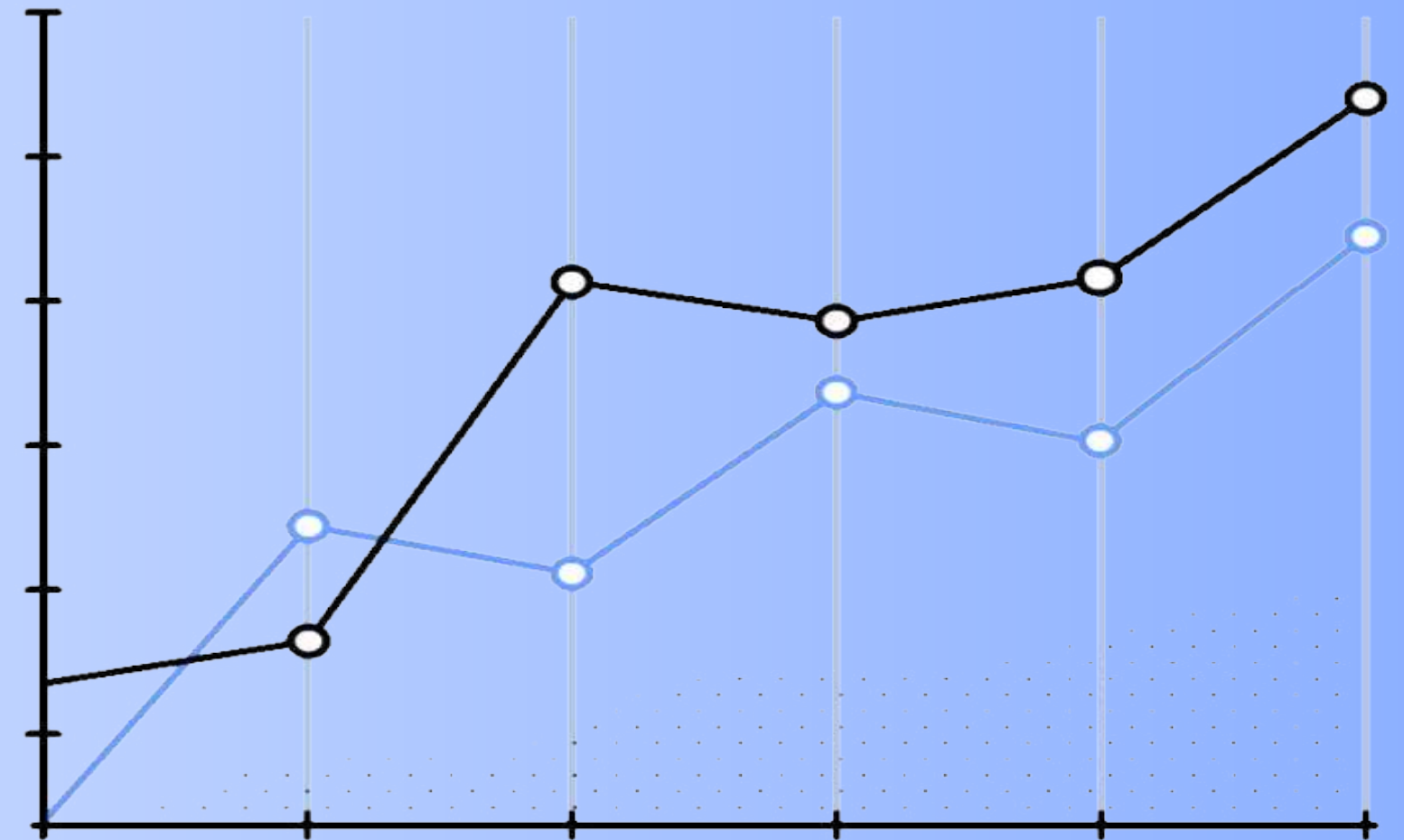
Prozessmanagement ist ohne Virtualisierung oder Containerisierung möglich, aber komplexer und weniger umfangreich als Docker Compose



PROZESSE

ERGEBNIS

Containerisierung ist besser für komplexe
Multi-Service-Umgebungen geeignet

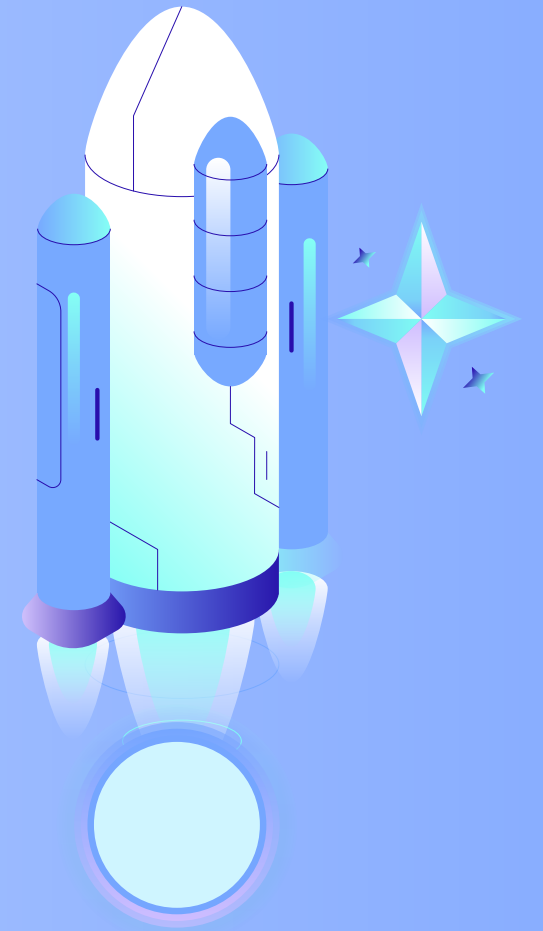
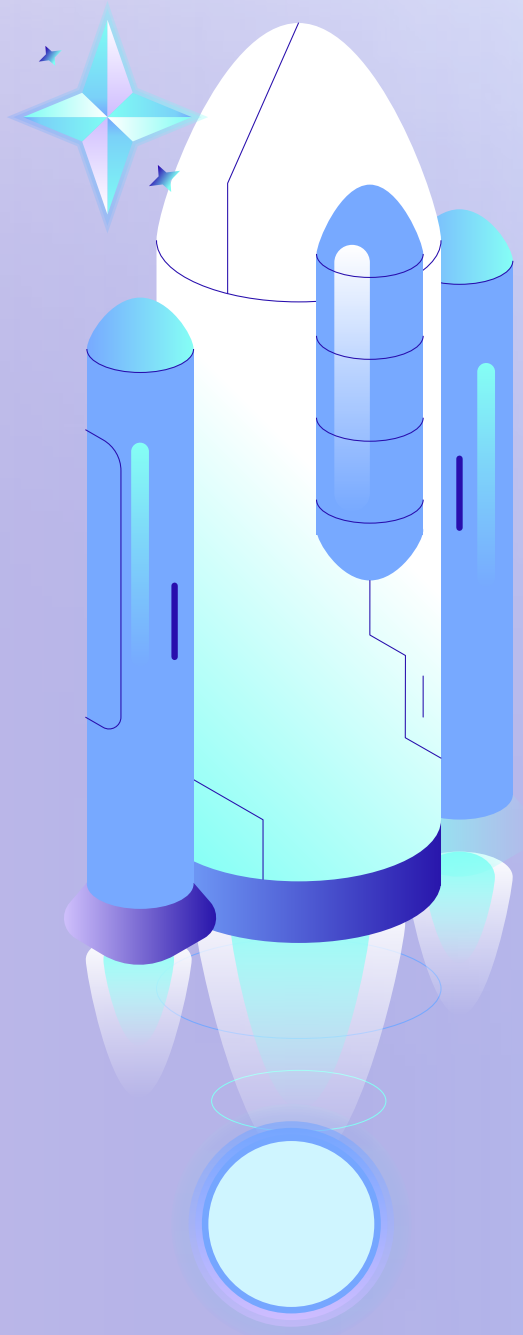




CONTAINER DEPLOYMENTS

Einsatz von Orchestrierungstools
wie Kubernetes für dynamisches
Skalieren

Hohe Skalierbarkeit

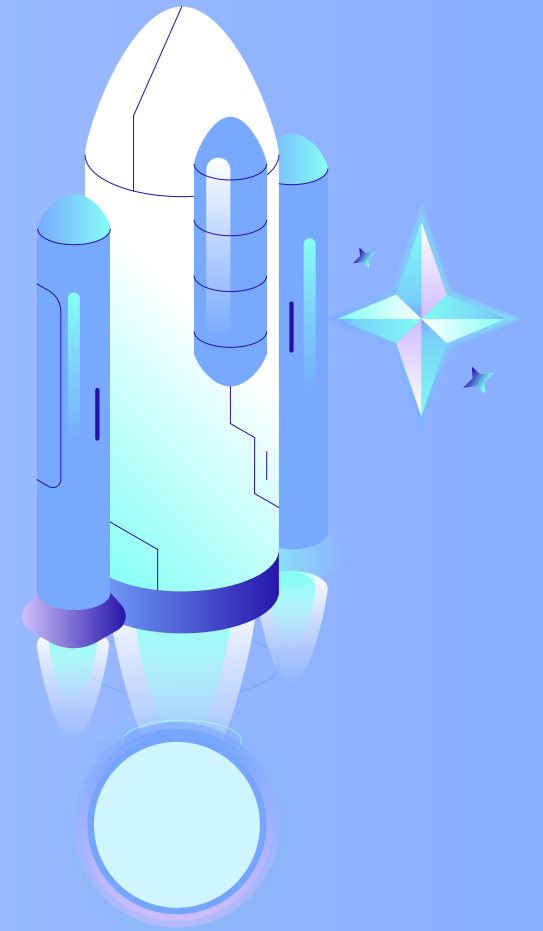
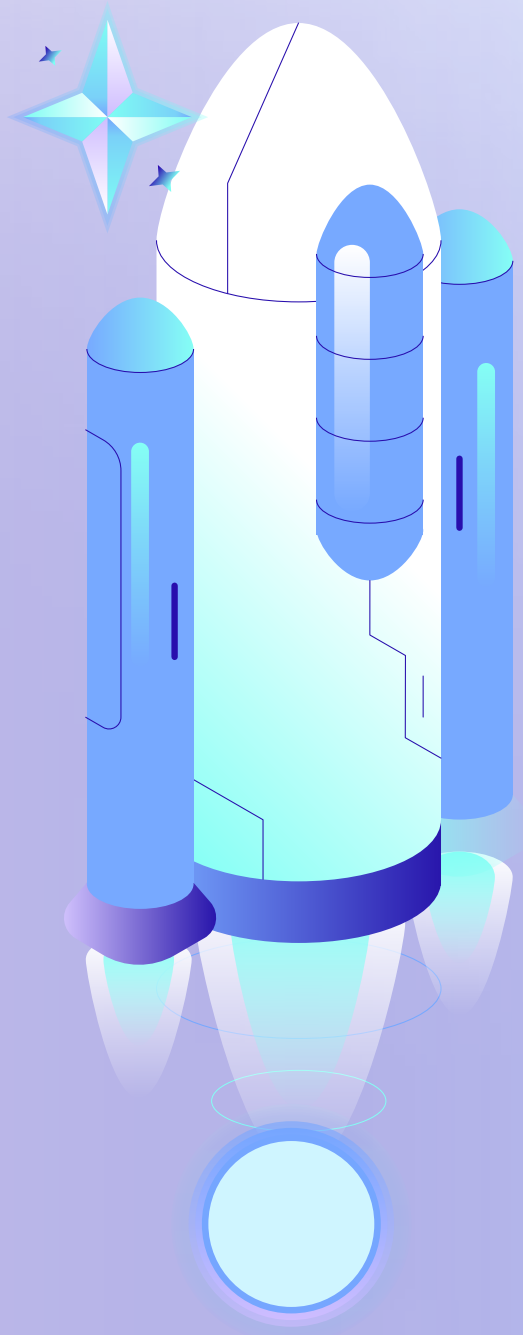




DERIVATION DEPLOYMENTS

Konsistente Deployments auf
unterschiedlichen Systemen

Skalierung erfordert manuelle
Intervention oder Automatisierungstools

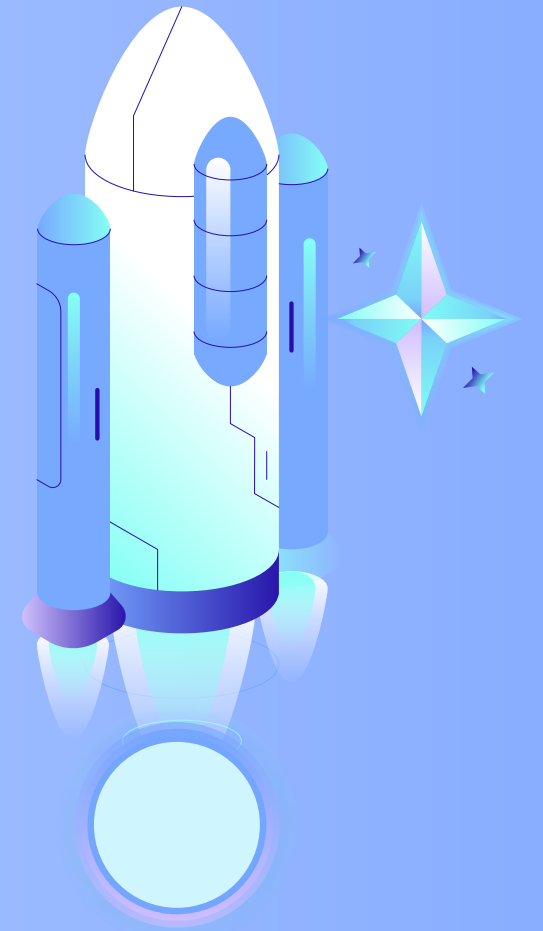
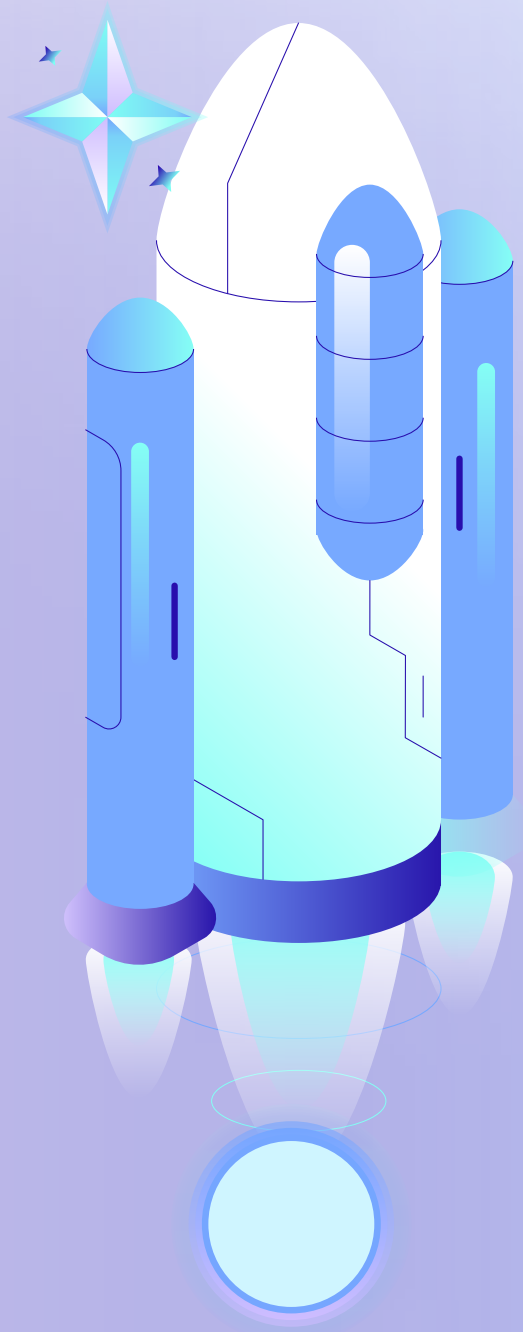


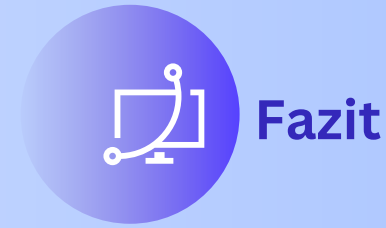


DEPLOYMENT ERGEBNISSE

Containerisierung bietet mehr
Automatisierung und Skalierbarkeit

Funktionales Paketmanagement
punktet mit reproduzierbaren und
schlanken Deployments



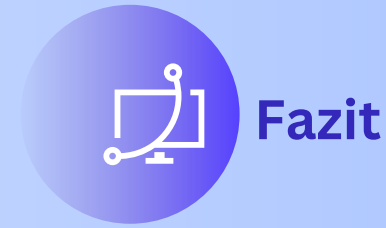


ERGEBNISSE

ZUSAMMENFASSUNG

Funktionales Paketmanagement zeigt deutliche Vorteile bei Reproduzierbarkeit, Konsistenz und Effizienz, während Containerisierung bei Skalierbarkeit und Orchestrierung dominiert.





STÄRKEN

FUNKTIONALES PAKETMANAGEMENTS

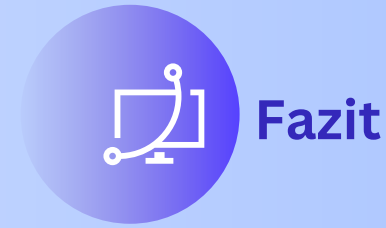
Exakte Kontrolle über Abhängigkeiten

Vermeidung der "Dependency Hell"

Reproduzierbare Umgebungen

Ideal für monolithische Anwendungen
und traditionelle Server





STÄRKEN

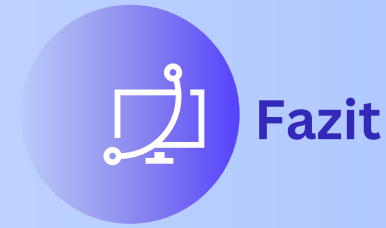
CONTAINERISIERUNG

Containerisierung isoliert Anwendungen effektiv

Schnelles Hochfahren neuer Instanzen

Glänzt bei der Verwaltung von Microservices durch Tools wie Kubernetes.





ANWENDUNGSFÄLLE

FUNKTIONALES PAKETMANAGEMENTS

Startups

Stabile Serverumgebungen

Projekte, die auf langfristige Konsistenz angewiesen sind

Entwicklungsumgebungen allgemein ➡

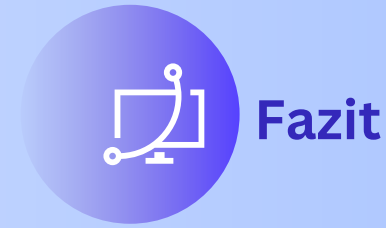


ANWENDUNGSFÄLLE

CONTAINERISIERUNG

Hochskalierbare, verteilte Systeme, die eine feingranulare Kontrolle über Ressourcen und schnelle Anpassung an Laständerungen benötigen.



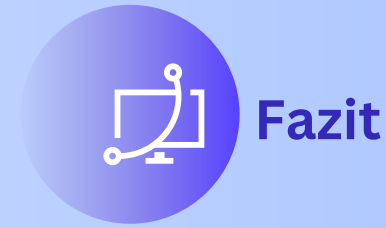


EMPFEHLUNG

ENTWICKLUNGSUMGEBUNGEN

Funktionales Paketmanagement eignet sich hervorragend für Entwicklungsumgebungen und sollte bevorzugt implementiert werden





EMPFEHLUNG

DEPLOYMENTS

Funktionales Paketmanagement für stabile, reproduzierbare Deployments ohne große Skalierungsanforderungen

Containerisierte Deployments eignen sich besser für dynamische und service-orientierte Anwendungen

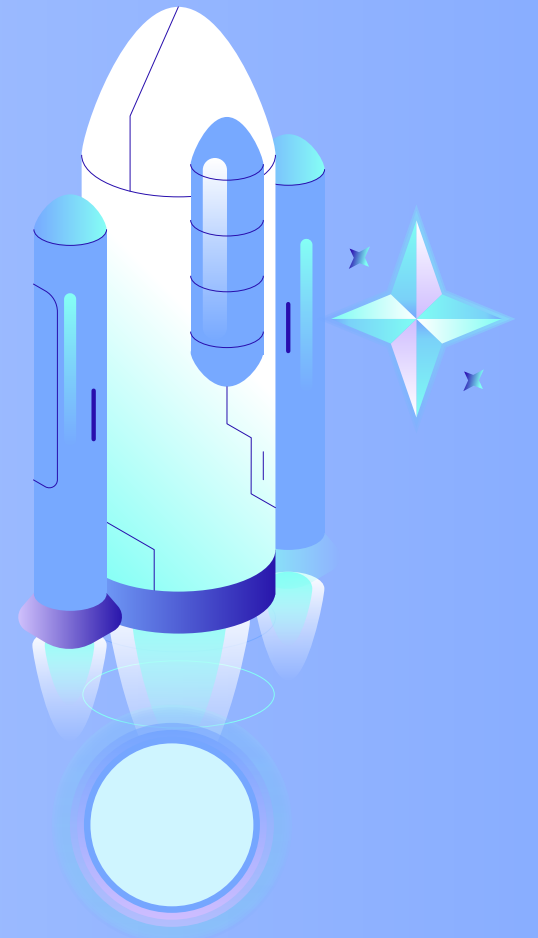
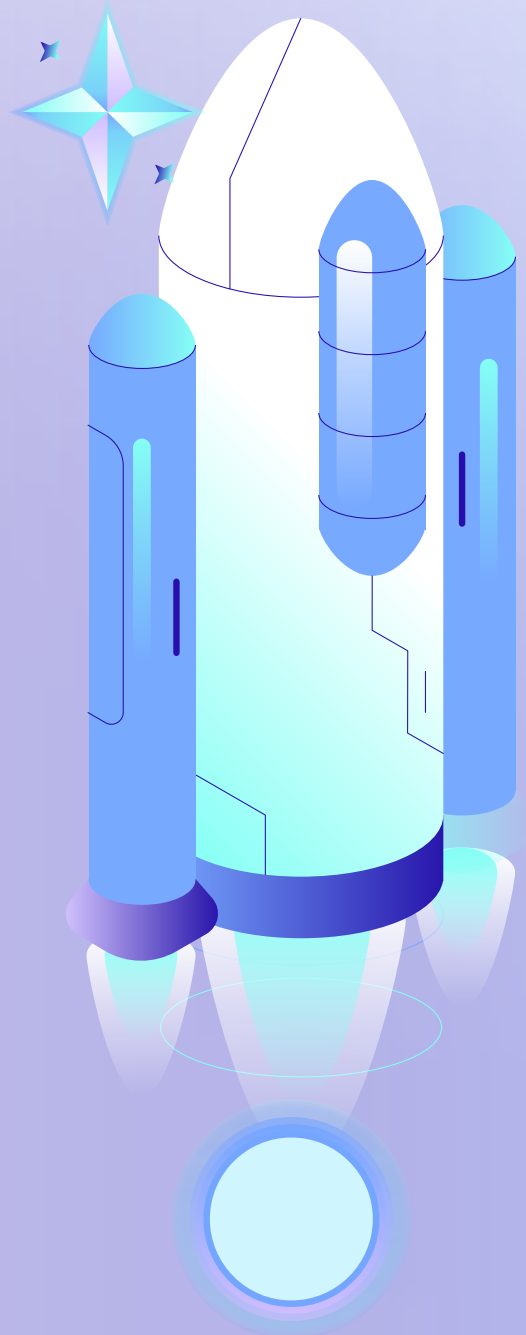




Analyse

PERSPEKTIVEN

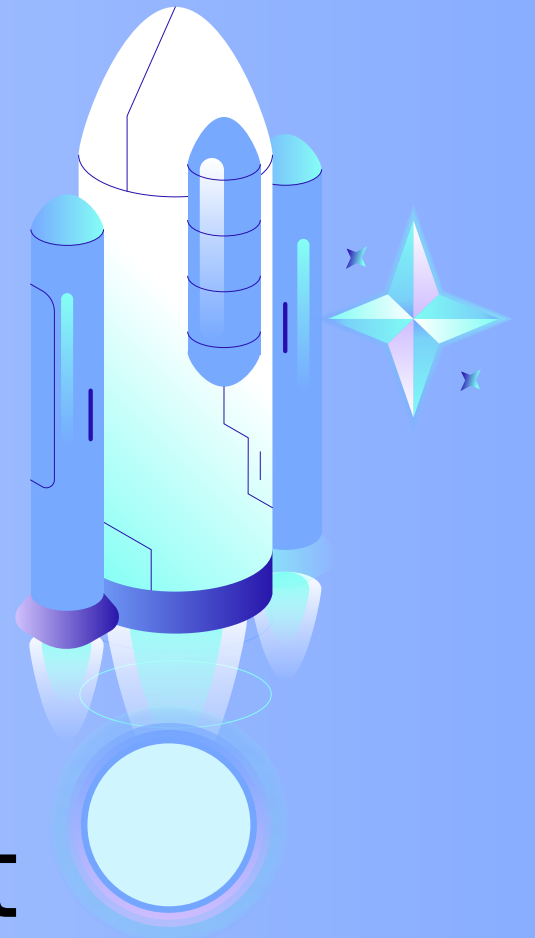
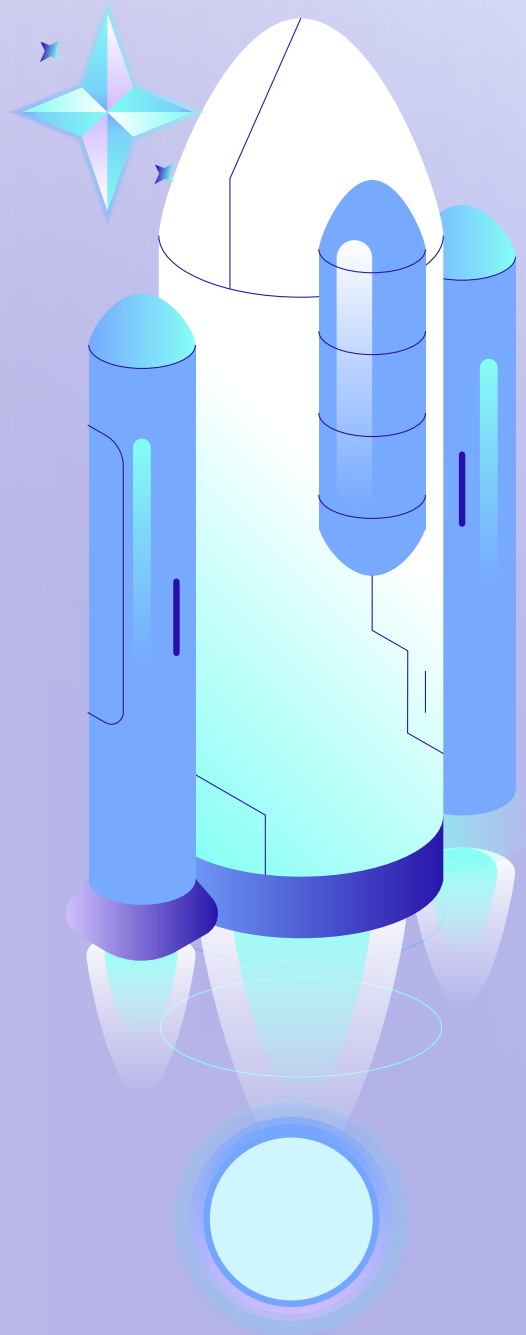
Hybride Ansätze könnten die Reproduzierbarkeit von funktionalem Paketmanagement mit der Flexibilität der Containerisierung kombinieren, um sowohl Stabilität als auch Skalierbarkeit zu gewährleisten.





IMPLIKATIONEN FÜR DIE PRAXIS

Für traditionelle Server, Monolithen und Startups ist funktionales Paketmanagement oft die bessere Wahl, da es weniger Overhead verursacht. Containerisierung zeigt ihre Stärken in komplexen, cloud-basierten Microservice-Architekturen.

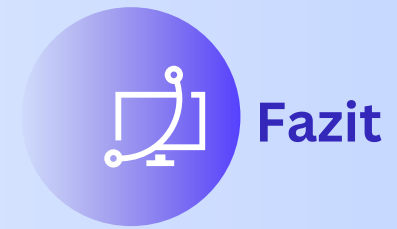




IMPLIKATIONEN FÜR DIE FORSCHUNG

Es besteht Forschungsbedarf, wie sich hybride Modelle umsetzen lassen, die die Vorteile beider Technologien nutzen, um eine optimierte Softwarebereitstellung zu ermöglichen.





VIELEN DANK!